COMSATS UNIVERSITY ISLAMABAD

Sahiwal Campus | Department of Computer Science

MOBILE APPLICATION DEVELOPMENT

FLUTTER & DART | COMPREHENSIVE LAB MANUAL

COURSE	ACADEMIC DETAILS
Course code	CSC303
Course title	Mobile Application Development
Credit hours	3(2,1)
Scheduled lab time	3 hours per week
Course prerequisite	CSC241 — Object Oriented Programming
Instructor	Muhammad Jamshaid Iqbal
Lab assessment	Assignments 25 | Midterm 25 | Final 50
Primary framework	Flutter / Dart — Android physical-device delivery

Student edition | CDF-aligned handbook | Revised November 2026

FIELD NOTE • Each solved Flutter task supplies complete app code and exact file placement. Cloud examples require the student's own generated Firebase configuration and service activation; no credentials are embedded.

COURSE & LABORATORY PROFILE

COMSATS University Islamabad | Sahiwal Campus | Department of Computer Science

Course code	CSC303
Course title	Mobile Application Development
Credit hours / practical time	3(2,1)	3 laboratory hours per week
Pre-requisite	CSC241 - Object Oriented Programming
Instructor	Muhammad Jamshaid Iqbal
Primary development tools	Flutter and Dart; Android Studio; Git and GitHub
Deployment target	Physical Android phone; emulator optional
Laboratory assessment	Assignments 25	Midterm 25	Final 50 = 100 marks

PURPOSE OF THIS MANUAL

This practical handbook accompanies CSC303 Mobile Application Development. It guides students from installation and source-control fundamentals to building, testing and presenting a multi-screen Android application using Flutter. The lab sequence follows the supplied course description and lecture-wise plan.

CDF LABORATORY LEARNING OUTCOME

Outcome	Expected student achievement
**CLO-5	Creating**	Develop advanced mobile applications with multiple screens and APIs having persistent storage in group and individual work.

LABORATORY COMPETENCIES

- Configure the Android development toolchain and deploy/debug Flutter apps on a real Android device.
- Use Git version history, SSH authentication, branches, issues, code review and pull requests for collaboration.
- Design adaptive interfaces, validate input, manage state and implement dependable navigation.
- Access REST services, serialize JSON and persist application data securely for offline use.
- Organize source code using a maintainable architecture and evaluate testability, performance and accessibility.

STUDENT LABORATORY RECORD

Student name	_____
Registration number	_____
Section / semester	_____
Academic session	_____

CONTENTS & LABORATORY SEQUENCE

Revised page numbers include new implementation and verification extensions.

Lab	Practical topic	Page
01	Professional setup, Git & GitHub workflow	8
02	Dart foundations and project anatomy	19
03	Widget tree, layouts and adaptive UI	25
04	Input forms, validation and local state	29
05	Navigation, routes and passing data	32
06	Scrollable lists, cards and feedback	35
07	AppBar, Drawer, bottom nav, tabs and themes	38
08	Shared state: Provider and Cubit/BLoC	42
09	Preferences and SQLite/Drift	44
10	HTTP REST, JSON and offline fallback	48
11	Firebase Authentication and sessions	52
12	Firestore, Storage and FCM	57
13	MVVM, repositories and testing	62
14	Rendering, animation and native channels	67
15	Integrated capstone and final project	71

PRELIMINARY READING & REVISION PAGES

Course and laboratory profile	2
How to use; marks and rubric	4
Revision guide	6
Workspace and environment contract	7
Release verification and change record	82

ADDITIONAL REVISED EXERCISES

GitHub portfolio workflow	18
Accessibility testing	28
Midterm evidence	41
Drift CRUD completion	47
API cache validation	51
Authentication journey	56
Firebase safeguards	61
ViewModel tests	66
Performance checks	70
Capstone integration and rubric	76

How to use this manual

This manual follows CSC303's five CDF units and the supplied Flutter lecture-wise plan. Unit I establishes tooling and framework structure; Unit II develops responsive UI, widgets, forms and navigation; Unit III covers advanced navigation and state management; Unit IV develops persistent storage, APIs and Firebase services; Unit V emphasizes architecture, performance, testing and the semester project. The CDF lab outcome (CLO-5) requires students to develop multi-screen applications with APIs and persistent storage individually and in groups.

Every lab supplies two worked alternatives, scenario questions, viva questions and a submission check. Labs 02-15 belong to independent Flutter application folders within the portfolio repository described in the revision guide. Tasks that both replace lib/main.dart are alternatives: commit or copy the first task before switching examples. Lab 01 deliberately uses its own starter repository for reviewed pull-request practice. A generated starter widget test may refer to MyApp; update it to match the current app before running flutter test. Never submit a knowingly failing test. New extension pages after selected labs define stronger validation and integration tasks.

Classroom conventions: Terminal commands appear without shell prompts and can be copied one line at a time. Lines starting with # in shell blocks are comments. Replace UPPERCASE placeholders only where account- or project-specific (GitHub username, device ID or SDK path). Windows filesystem PATH examples and Firebase console security-rule blocks are reference/configuration snippets, NOT commands to paste into a shell. Use Git Bash for the Git commands; the optional PowerShell examples are labelled separately. A generated Flutter starter widget test commonly refers to MyApp; after changing lib/main.dart, replace or remove that outdated test before flutter test.

Lab marks and evaluation

Component	Marks	Evidence
Assignments / practical portfolio	25	Five assessed checkpoints × 5 marks; submitted code, Git history, screenshot, technical note
Lab midterm practical	25	Individual timed implementation and demonstration, normally after Lab 07
Lab final / semester project	50	Working Android application, code architecture, APIs, persistence, Git collaboration, report and viva
TOTAL	100	Lab grading, separate from the theory component

Suggested 25-mark assignment checkpoints: Assignment 1 (Lab 01 Git + collaboration, 5); Assignment 2 (Lab 04 forms and UI, 5); Assignment 3 (Lab 07 advanced navigation, 5); Assignment 4 (Lab 10 networking and offline data, 5); Assignment 5 (Lab 13 architecture and tests, 5). The instructor may reassign checkpoint dates while preserving 25/25/50.

Assessment rubrics

Instrument	Criteria and allocation
Each assignment / 5	Correctness 2; evidence and reproducibility 1; Git hygiene 1; explanation/viva 1.
Midterm / 25	Working features 12; UI/events 5; code structure 4; debugging and viva 4.
Final project / 50	Functionality 15; API and persistence 10; UI/accessibility 7; architecture/tests 6; Git process 5; demo and viva 7.

Practical schedule and alignment

Lab	Topic	CDF unit
01	Git, GitHub SSH, contribution protocol,	1

Lab	Topic	CDF unit
	Android Studio and physical device	
02	Dart foundations, project structure and resources	1
03	Widgets, composition and responsive layout	2
04	Forms, input controls and setState	2
05	Multi-screen navigation and routes	2–3
06	Lists, cards, dialogs and scrolling	3
07	Drawer, bottom navigation, tabs and themes	3
08	Provider and Cubit/BLoC state management	3–4
09	Shared preferences and Drift / SQLite	4
10	REST API and offline-first caching	4
11	Firebase email/password authentication	4
12	Firestore, Cloud Storage and FCM	4
13	MVVM, repositories, dependency injection and tests	5
14	Performance, animations and Android MethodChannel	3–5
15	Integrated capstone implementation and final submission	2–5

Standard lab submission

Upload a GitHub repository link and a release/commit SHA; submit a README with prerequisites, setup commands, screenshots of the app on a physical Android device, a 100–150 word explanation of the algorithms/state used, and brief test evidence (flutter analyze or flutter test). Commit actual source files, not build/ or .dart_tool/. Never upload SSH private keys, Firebase service-account JSON, user credentials or signing keystores.

CSC303 / NOVEMBER 2026 REVISION

REVISION GUIDE | Read before Lab 01

Edition: November 2026 teaching revision. The original 15-lab sequence, solved tasks, scenario questions, and official 25/25/50 marking allocation are retained. This edition adds implementation guidance, explicit acceptance checks, stronger security boundaries and integrated-project planning.

Which instructions govern the revised edition?

• **Repository:** keep one GitHub portfolio repository with independent Flutter applications in `labs/` and a dedicated `semester_project/`. A lab project can be created directly inside the portfolio; see the next page. The original Lab 01 csc303-mad repository is an isolated Git exercise, not a second required portfolio submission.

• **Worked tasks:** when Task 1 and Task 2 both say `lib/main.dart`, they are alternative examples, not simultaneous source files. Save each working version in a task branch or separate app directory before replacing a file.

• **Tests:** replace an obsolete starter `test/widget_test.dart` with a relevant test. Do not make removing all tests a normal substitute for testing. The new extensions add test cases for repositories and bad network data.

• **Cloud features:** Firebase Storage requires a Blaze project. Students must not activate billing without written instructor authorization; equivalent no-billing demonstrations are listed in the Lab 12 revision.

• **Final project:** the two Lab 15 solved samples are teaching components. Neither, by itself, meets the full rubric for the 50-mark final deliverable. The new capstone extension defines the integration path.

Three-hour laboratory contract

Phase	Time	Deliverable
Pre-lab (outside scheduled session)	Variable	Install tools, fetch packages, read instructions; verify access
Guided demonstration	30 min	Instructor explains concepts, runs the minimal reference path
Implementation and fault tests	110 min	Student completes a defined subset and records outcome
Review, viva and evidence	40 min	Push code, show acceptance checks, document unresolved issues

The timings are a proposed classroom structure rather than a change to official contact hours. Labs 01, 11 and 12 require pre-lab preparation; no student should be penalized for unapproved billing or an instructor-controlled cloud service being unavailable.

Evidence and status notation

• **RUN:** executed on an Android phone. **PASS:** automated tests actually executed. **STATIC:** inspected but not executed. **BLOCKED:** external setup, network or billing prevented a test. Screenshot-only evidence must not be labeled PASS.

Revision validation note: these editorial changes were statically checked for internal consistency and PDF layout. The complete set of Flutter labs was not executed end-to-end in a pinned SDK environment. A departmental release should perform the verification matrix below before calling the edition build-certified.

CSC303 / NOVEMBER 2026 REVISION

REPRODUCIBLE LAB WORKSPACE | Student protocol

The official toolchain versions move over time. Record an actual, instructor-approved stable Flutter version and dependency lockfile at semester start; do not silently upgrade one student project mid-assessment.

Environment and device acceptance

```
flutter --version
dart --version
flutter doctor -v
flutter doctor --android-licenses
adb devices
flutter devices
```

Record: operating system, Flutter/Dart SDK versions, Android Studio version, Android compile SDK/API level, JDK detected by Flutter, device model/Android release, and the date. Instructor publishes one tested environment fingerprint and a fallback procedure for platform-specific toolchain issues.

One portfolio repository, multiple Flutter apps

```
csc303-portfolio/
  README.md
  CONTRIBUTING.md
  .github/PULL_REQUEST_TEMPLATE.md
  labs/mad_lab02/          # pubspec.yaml is HERE
  labs/mad_lab03/
  labs/mad_lab13_task1/
  labs/mad_lab13_task2/
  semester_project/      # independent assessed app
```

```
mkdir csc303-portfolio
cd csc303-portfolio
git init -b main
mkdir -p labs
cd labs
flutter create --platforms=android mad_lab02
cd mad_lab02
flutter pub get
flutter analyze
flutter run
```

Run `flutter` and `dart` commands inside each application folder (where its `pubspec.yaml` lives). Git commands work from the portfolio root or a subdirectory. Prefer reviewed, issue-linked branches for assessed changes.

General verification commands

```
dart format lib test
flutter analyze
flutter test
flutter run -d ANDROID_DEVICE_ID
flutter build apk --debug
git status --short
git rev-parse HEAD
```

A project may need code generation before analysis (Lab 09) or a generated `firebase_options.dart` (Labs 11–12). Never commit private keys, passwords, service-account JSON or signing keystores. An application `pubspec.lock` should normally be committed for repeatability.

Suggested per-lab submission manifest

Lab / task: _____ **Flutter version:** _____ **Device:** _____ **Commit:** _____
Run: PASS / BLOCKED **Analyze:** PASS / FAIL **Tests:** PASS / N/A **Evidence:** _____

LAB 01 Professional setup, Git & GitHub contribution workflow

Unit 1 • 3-hour guided laboratory • CLO-5

Learning goal. Install the complete Android development toolchain, deploy an app to a REAL Android phone, authenticate GitHub with an SSH key, and contribute through reviewed pull requests.

Core concepts. Git working tree / staging / commits / branches / remotes; ED25519 authentication; Android SDK Manager; Flutter SDK, platform tools and ADB; collaboration and reviews.

Before you start. A Windows 10/11 laptop is the illustrated host. Linux/macOS commands are noted where different. Have a GitHub account, internet connection, USB data cable and a physical Android phone.

A. Install Git, Android Studio and the complete Android toolchain

STEP 01 | Install Git and Git Bash

Download Git for Windows from https://git-scm.com/downloads. Install Git Bash with OpenSSH available, then verify the two programs in Git Bash. Use your normal account instead of running routine Git work as administrator.

STEP 02 | Install Android Studio

Download the current stable Android Studio from https://developer.android.com/studio. Complete the Setup Wizard, choose the standard Android SDK setup, accept the licenses and allow the downloads to finish.

STEP 03 | Download an Android SDK platform

Open Android Studio Welcome screen > More Actions > SDK Manager (or Tools > SDK Manager from an open project). On the SDK Platforms tab, select a supported stable Android API platform and click Apply.

STEP 04 | Install the required SDK Tools

In SDK Manager > SDK Tools, select the required tools listed in the table below, then click Apply. The emulator is optional because these labs use a physical Android phone. NDK (Side by side) and CMake are only needed if a plugin or build explicitly requests them.

Component	Where to select	Required for phone?
Android SDK Platform	SDK Platforms tab	Yes
Android SDK Platform-Tools (adb)	SDK Tools tab	Yes
Android SDK Build-Tools	SDK Tools tab	Yes
Android SDK Command-line Tools (latest)	SDK Tools tab	Yes
Android Emulator	SDK Tools tab	No - optional
Google/OEM USB Driver (Windows)	SDK Tools / manufacturer	Only if needed
NDK (Side by side), CMake	SDK Tools tab	Only if project needs them

STEP 05 | Locate the Android SDK and enable Flutter plugins

Copy the Android SDK Location shown in SDK Manager; do not guess the installed path. On Windows, a common location is shown below. In Android Studio, open Settings > Plugins > Marketplace, install Flutter, accept the Dart plugin installation and restart the IDE.

```
%LOCALAPPDATA%\Android\Sdk    (common default; verify in SDK Manager)
```

Plugin walkthrough: Flutter and Dart in Android Studio

PLUGIN 1 | Open the Marketplace

Windows/Linux: File > Settings (Ctrl+Alt+S) > Plugins > Marketplace. macOS: Android Studio > Settings > Plugins. Ensure the Marketplace tab, not Installed, is selected.

PLUGIN 2 | Install the Flutter plugin

Search for Flutter, select the plugin published for Flutter development, and click Install. Approve any permission or compatibility dialog shown by your installed IDE version.

PLUGIN 3 | Install the Dart plugin

When Android Studio asks to install or enable the Dart plugin as a dependency of Flutter, select Install/Enable. If no prompt appears, open Plugins > Installed and confirm BOTH Flutter and Dart are enabled. If Dart is missing, search for Dart in Marketplace and install it.

PLUGIN 4 | Restart and locate Flutter settings

Restart Android Studio. Verify Flutter and Dart appear enabled in Plugins > Installed. The New Flutter Project option should now appear. Some IDE versions also expose Languages & Frameworks > Flutter to set the Flutter SDK location.

PLUGIN 5 | Distinguish the plugin from the SDK

The Android Studio plugins add editor integration, project templates, code completion, debugging and the Flutter Inspector. They DO NOT install the Flutter command-line SDK. Install Flutter separately in Step 06. The Flutter SDK includes the Dart SDK, so a second standalone Dart SDK is NOT required for these exercises.

PLUGIN + SDK VERIFICATION | Run AFTER Step 06 in a new terminal

```
flutter --version
dart --version
flutter doctor -v
```

Successful check: Android Studio shows Flutter AND Dart under Installed plugins, both commands print versions, and flutter doctor recognizes the Android toolchain. If the plugin is installed but flutter is not a recognized command, fix the Flutter SDK PATH rather than reinstalling the plugin.

STEP 06 | Install the Flutter SDK and update PATH

Download the stable Flutter SDK from https://docs.flutter.dev/install/manual. Extract the SDK to a user-owned directory (for example C:\src\flutter), rather than Program Files. Add its bin directory to your user PATH and restart terminals. Installing only the Android Studio plugin does NOT install the Flutter SDK.

```
C:\src\flutter
C:\src\flutter\bin    (add this directory to your user PATH)
```

STEP 07 | Validate the toolchain and accept Android licenses

Run Flutter Doctor and accept Android SDK licenses. Confirm that Android toolchain and Android Studio are detected; the connected phone will be checked in the next section. If needed, point Flutter to the exact Android SDK location shown in SDK Manager.

```
flutter doctor -v
flutter doctor --android-licenses
# Only if Flutter cannot locate your actual installed Android SDK:
flutter config --android-sdk "ACTUAL_SDK_PATH"
```

STEP 08 | Create a Flutter project

In Android Studio select New Flutter Project > Flutter, choose the installed Flutter SDK, name your project and select Android. You can also create a project from a terminal. Use a reverse-domain organization ID for real projects.

Guided walkthrough: create your first app with Android Studio

WIZARD 1 | Launch the correct project wizard

From the Welcome screen choose New Flutter Project. If another project is open, select File > New > New Flutter Project. On IDE versions that first show a Generators list, select Flutter; choose an Application rather than a Plugin or Module.

WIZARD 2 | Select the Flutter SDK DIRECTORY

When asked for Flutter SDK path, browse to the folder containing bin/, packages/ and other Flutter SDK files (for example C:\src\flutter). Do not select C:\src\flutter\bin and do not select the Android SDK. The IDE may display a Next button after it locates a valid SDK.

WIZARD 3 | Enter project metadata carefully

Use the following worked example. Some Android Studio versions show slightly different field names. A project name uses lowercase letters, numbers and underscores; it is NOT the visible app name on the phone.

Wizard field	Example / explanation
Project type	Flutter Application / Application, not Module or Plugin.
Project name	mad_wizard_demo (valid Dart package name; avoid spaces and hyphens).
Description	CSC303 practice: first Android Flutter application.
Project location	Select a writable folder under your own account; ensure the final project directory is mad_wizard_demo, not a duplicate nested folder.
Organization	edu.cui.sahiwal (classroom example used in this manual; use your own unique reverse-domain identifier for publishing).
Platforms	Select Android. iOS, web and desktop are unnecessary for these phone-focused labs.
Android language	Keep Kotlin if Android Studio offers this option.

WIZARD 4 | Finish and let Android Studio initialize the project

Click Create or Finish. Wait while files are generated and Dart/Flutter packages are resolved. Android Studio may also index code and download Gradle dependencies on first run; this needs an internet connection.

WIZARD 5 | Open the actual Flutter project root

In the Project tool window, change the tree-view selector to Project or Project Files if you only see the Android view. Verify pubspec.yaml, lib/main.dart, android/ and test/ appear together under mad_wizard_demo. A directory containing only android/ is not the Flutter project root.

WIZARD 6 | Select and run on your physical Android phone

Connect the phone and approve its USB debugging authorization (Part B below). Select the phone in Android Studio device dropdown. Open lib/main.dart and click the green Run triangle; the starter counter screen should launch on the PHONE. You can alternatively run flutter run from the project-root Terminal.

WIZARD 7 | Confirm your successful first build

Tap the starter counter + button and confirm its displayed count changes. Edit a displayed label, save, and trigger Hot Reload while the app runs. Record the connected device name, one screen capture and whether the toolchain passes flutter doctor.

EQUIVALENT COMMAND-LINE METHOD | Use a DIFFERENT name if the wizard app already exists

```
flutter create --platforms=android --org edu.cui.sahiwal mad_cli_demo
cd mad_cli_demo
flutter pub get
flutter devices
flutter run
```

After the code: flutter create generates the project with an Android platform target; cd enters the folder containing pubspec.yaml; flutter pub get resolves the dependencies; flutter devices lists available targets; flutter run builds and installs the app on your selected connected phone. If several devices are listed, the tool prompts you to choose one, or you can use flutter run -d followed by the exact ID it shows.

Important distinction: the wizard and terminal are ALTERNATIVE ways to create an app. Do NOT run flutter create into an already populated directory simply to repeat the wizard. Solved Task 1 later in this lab intentionally uses flutter create . inside the newly initialized csc303-mad Git repository; that is a separate, documented workflow.

First-run troubleshooting: identify the failing layer

Symptom	Likely cause and corrective action
New Flutter Project is missing	Enable/install both IDE plugins and restart; installing Flutter SDK alone does not add IDE actions.
Flutter SDK path rejected	Point to the SDK ROOT directory (the one containing bin/), not bin/ or the Android SDK.
flutter: command not found	Add Flutter SDK bin/ to user PATH; restart all terminals and Android Studio.

Symptom	Likely cause and corrective action
No physical phone in device list	Check USB debugging, phone authorization prompt and DATA cable; run adb devices and flutter devices.
adb: unauthorized	Unlock phone, approve the host RSA fingerprint, then retry; revoke authorizations and reconnect if needed.
Android licenses not accepted	Run flutter doctor --android-licenses, accept all agreements, rerun flutter doctor -v.
Gradle or SDK download fails	Check stable network, Android SDK installation, free disk space and Gradle errors in the Build tool window.
Only android/ appears in Project pane	Switch IDE Project pane from Android to Project / Project Files to reveal lib/, test/ and pubspec.yaml.

Student checkpoint: show the instructor (1) both installed plugins, (2) the Flutter SDK root, (3) an Android SDK platform and Platform-Tools, (4) a listed and authorized phone, and (5) an app running on the phone. Official reference: https://docs.flutter.dev/reference/create-new-app.

FINAL CHECK | Run these commands individually in Git Bash / terminal

```
git --version
ssh -V
flutter --version
flutter doctor -v
flutter doctor --android-licenses
flutter devices
```

B. Configure a physical Android phone — no emulator required

STEP 01 | Enable Android developer options

On the Android phone, open Settings > About phone, tap Build number approximately seven times (the menu can vary by manufacturer), then open Developer options and enable USB debugging. Disable debugging when it is no longer needed.

STEP 02 | Connect and authorize the computer

Connect with a USB DATA cable, unlock the phone, select File Transfer (MTP) if prompted, and approve the displayed RSA fingerprint only if it belongs to your computer.

STEP 03 | Install an OEM driver if Windows cannot detect the phone

If Windows Device Manager reports an unknown USB phone, install the manufacturer-supplied OEM USB driver. Android SDK Platform-Tools supplies adb, but the Windows hardware driver is a separate component on some devices.

STEP 04 | Confirm device authorization with ADB

Run adb devices and check the status. The phone should show device, NOT unauthorized. Then run flutter devices and select the phone in Android Studio or use flutter run with its device ID.

STEP 05 | Deploy, hot reload and build the APK

The first build may download Gradle and Android libraries, so remain connected. Press r in the terminal for hot reload while running. When ready, build a release APK using flutter build apk --release; the output is placed in build/app/outputs/flutter-apk/.

DEVICE CHECK / RUN / APK

```
adb devices
flutter devices
flutter doctor -v
# Use the ID printed by flutter devices when more than one target is attached.
flutter run -d ANDROID_DEVICE_ID
# Build an installable release package after your app works.
flutter build apk --release
```

FIELD NOTE ● If adb is not found, use Android Studio Terminal after SDK Platform-Tools installation or add the actual SDK platform-tools directory to PATH. For a device marked unauthorized, unplug, revoke USB debugging authorizations on the phone, reconnect and approve the new prompt.

C. Git essentials: explain the workflow before pushing

Concept	Technical meaning / recommended command
Working tree	Files currently edited; inspect with git status and git diff.
Index (staging)	Proposed next snapshot; git add <path>, git diff --cached.
Commit	Local immutable history snapshot; git commit -m "...".
Branch	Named reference to a line of commits; git switch -c feat/task-name.
Remote	Reference to hosted repository; git remote -v, git fetch origin.
Push / pull	Push publishes local commits; pull fetches + integrates remote changes.
Merge / rebase	Integrate change history; rebase rewrites local commit IDs—never rewrite shared history casually.
Ignore rules	Git .gitignore prevents generated files, local secrets and build artifacts from being staged.

D. Set up GitHub authentication using ssh-keygen

GitHub SSH authentication uses two files: a private key, which MUST remain on your computer, and a .pub public key, which you register with GitHub. Use a passphrase to protect the private key. Follow the steps in order, and never email, upload or commit your private key.

STEP 01 | Set your Git identity

This identity appears in commit metadata; use the email associated with your GitHub account.

```
git config --global user.name "Your Full Name"
git config --global user.email "YOUR_GITHUB_EMAIL@example.com"
git config --global init.defaultBranch main
```

STEP 02 | Check for an existing key, then create an ED25519 key

Use an existing secure key if appropriate. Otherwise run ssh-keygen; accept the default path unless it would overwrite an existing key, and set a strong passphrase.

```
ls -al ~/.ssh
ssh-keygen -t ed25519 -C "YOUR_GITHUB_EMAIL@example.com"
```

STEP 03 | Start ssh-agent and load your PRIVATE key locally

The agent remembers the unlocked key during your session. It does not upload the key to GitHub.

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

STEP 04 | Display your PUBLIC key for GitHub

Copy the ENTIRE line printed below (normally starts with ssh-ed25519). On GitHub: avatar > Settings > SSH and GPG keys > New SSH key > Authentication key. Paste the public key, set a descriptive device title and save.

```
cat ~/.ssh/id_ed25519.pub
```

STEP 05 | Test the GitHub SSH connection

On the first connection, verify the GitHub host fingerprint against its official documentation before accepting it. A greeting containing your GitHub username means SSH authentication succeeded. GitHub offers no shell, so the test can end with exit code 1.

```
ssh -T git@github.com
```

SECURITY CHECK | Private key stays local; upload ONLY the .pub key

```
# NEVER upload ~/.ssh/id_ed25519 to GitHub.
# Upload ONLY ~/.ssh/id_ed25519.pub as an Authentication key.
```

If ssh -T reports permission denied (publickey), check whether the correct key is loaded with ssh-add -l, whether the .pub file was registered on the correct GitHub account, and whether your Git remote begins with git@github.com:. Never solve this by sharing private keys.

ALTERNATIVE WINDOWS POWERSHELL — run first two lines with administrator privileges if necessary

```
Get-Service -Name ssh-agent | Set-Service -StartupType Manual
Start-Service ssh-agent
ssh-add "$HOME/.ssh/id_ed25519"
Get-Content "$HOME/.ssh/id_ed25519.pub"
```

A passphrase protects the private key at rest. The agent remembers unlocked key credentials for the current session. If Git Bash and Windows SSH-agent conflict, configure Git to use Windows OpenSSH with git config --global core.sshCommand "C:/Windows/System32/OpenSSH/ssh.exe".

Solved task 1 | Initialize, branch, commit and push a repository

Problem and objective. Create an empty GitHub repository named csc303-mad (do NOT initialize it with a README on GitHub). In Git Bash, create the matching LOCAL folder and generate a real Flutter application in its root before your first commit. Replace YOUR_USERNAME with your GitHub username. A private repository is acceptable if the instructor has reviewer access.

GIT BASH — project setup

```
mkdir csc303-mad
cd csc303-mad
git init -b main
git config --local user.name "Your Full Name"
# Create an Android Flutter app IN the Git repository, not as a sibling.
# A Dart package name cannot contain a hyphen, hence --project-name.
flutter create --platforms=android --project-name csc303_mad --org edu.cui.sahiwal .
printf "# CSC303 Flutter Laboratory\n\nFlutter Android coursework.\n" > README.md
# APPEND to Flutter's generated .gitignore; do NOT overwrite it.
printf "\n# CSC303 local-only secrets\n.env\n*.jks\n*.keystore\n" >> .gitignore
git status
git add .
git diff --cached --stat
git commit -m "chore: initialize Flutter project"
git remote add origin git@github.com:YOUR_USERNAME/csc303-mad.git
git push -u origin main
# Make a reviewable docs branch instead of editing main directly.
git switch -c docs/lab01-readme
printf "\n## Lab 01\nGit and Flutter setup verified.\n" >> README.md
git add README.md
git commit -m "docs: document Lab 01 setup"
git push -u origin docs/lab01-readme
```

How the solution works. git init builds the repository, but flutter create is what makes it a runnable Android app. The folder is named csc303-mad for GitHub while --project-name sets the legal Dart package name csc303_mad. Flutter generates pubspec.yaml, lib/, android/ and the initial test/ directory; its generated .gitignore must be preserved. Staging and committing snapshot the code; the SSH remote connects that snapshot to GitHub. The documentation branch allows review without direct edits to main.

Expected output. The repository root contains pubspec.yaml, lib/main.dart, android/ and README.md. GitHub shows main plus docs/lab01-readme; open a pull request from the documentation branch into main. The starter Flutter app is a genuine buildable project.

Check your work. Run git log --oneline --decorate -3, git status and git remote -v; the working tree should be clean.

Solved task 2 | Create, build and install the first Flutter app on the phone

Problem and objective. Use the SAME csc303-mad Flutter repository from Solved Task 1. Return to main, create a separate feature branch and replace lib/main.dart with the complete counter app below. Replace the generated starter test with the matching test below. Deploy to the physical phone, run tests and open a pull request for the feature.

GIT BASH / TERMINAL — from the csc303-mad repository root

```
# Begin IN the csc303-mad folder made in Task 1.
git switch main
```

```
git pull --ff-only origin main
git switch -c feat/first-phone-counter
flutter pub get
flutter devices
```

lib/main.dart — complete copy-paste file

```
import 'package:flutter/material.dart';

void main() => runApp(const FirstPhoneApp());

class FirstPhoneApp extends StatelessWidget {
  const FirstPhoneApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'My First Android App',
      theme: ThemeData(colorSchemeSeed: Colors.teal, useMaterial3: true),
      home: const FirstPhonePage(),
    );
  }
}

class FirstPhonePage extends StatefulWidget {
  const FirstPhonePage({super.key});
  @override
  State<FirstPhonePage> createState() => _FirstPhonePageState();
}

class _FirstPhonePageState extends State<FirstPhonePage> {
  int taps = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('CSC303 • My Android phone')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Icon(Icons.phone_android, size: 70),
            const SizedBox(height: 16),
            Text('Button tapped $taps times',
                style: Theme.of(context).textTheme.headlineSmall),
            const SizedBox(height: 12),
            FilledButton(
              onPressed: () => setState(() => taps++),
              child: const Text('Tap on my phone'),
            ),
          ],
        ),
      ),
    );
  }
}
```

test/widget_test.dart — replace the generated starter test

```
import 'package:flutter_test/flutter_test.dart';
import 'package:csc303_mad/main.dart';

void main() {
  testWidgets('counter increments when button is tapped', (tester) async {
    await tester.pumpWidget(const FirstPhoneApp());
    expect(find.text('Button tapped 0 times'), findsOneWidget);
    await tester.tap(find.text('Tap on my phone'));
    await tester.pump();
    expect(find.text('Button tapped 1 times'), findsOneWidget);
  });
}
```

TERMINAL — run from csc303-mad after saving BOTH Dart files

```
dart format lib test
flutter analyze
flutter test
flutter run
# Test on your phone; press q to quit flutter run before continuing.
flutter build apk --debug
# Commit the app AND the corrected widget test on the feature branch.
git add lib/main.dart test/widget_test.dart
git commit -m "feat: add first Android counter and widget test"
git push -u origin feat/first-phone-counter
# Open a GitHub PR from feat/first-phone-counter to main.
```

How the solution works. MaterialApp configures the app, Scaffold supplies the page structure, and StatefulWidget holds the mutable tap count. setState schedules a redraw after each press. The widget test checks the counter before and after tapping; it replaces the generated test that still expects the old MyApp class. Git records the app and test together on a feature branch, enabling review before merge.

Expected output. The physical phone shows an app bar, phone icon, counter and working button. flutter analyze and flutter test complete successfully. The feature branch is visible on GitHub and can be opened as a reviewed pull request.

Check your work. Run flutter analyze and flutter test; tap five times on your Android phone. Check git status, ensure the new test is committed, and demonstrate a pull request with both app code and test in its Files changed tab.

Assignment 1 — mandatory complete contribution guide (5 marks)

Build a shared CSC303 Flutter repository with a protected main branch and use this workflow for every later lab. The students must commit README.md, CONTRIBUTING.md and the pull request template shown below. The following guide is both the project policy and a ready-to-copy contribution file.

README.md explains how to run the project; CONTRIBUTING.md defines the team rules; .github/PULL_REQUEST_TEMPLATE.md ensures every contribution includes reproducible checks and screenshots. Create each file at the exact location written above its example.

CONTRIBUTING.md — save in repository root (replace values where indicated)

```
# Contributing to CSC303 Flutter Project

## 1. Scope
Contribute Flutter/Dart, Android configuration, tests and docs only.
Discuss substantial features in a GitHub Issue before starting.
Never commit passwords, SSH private keys, service accounts or keystores.

## 2. Local setup
1. Fork the repository if you do not have write access.
2. Clone via SSH: git clone git@github.com:YOUR_USERNAME/csc303-mad.git
3. cd csc303-mad
4. flutter doctor -v; flutter pub get; flutter run

## 3. Branches and commit messages
Create a branch from the latest main:
  git switch main
  git pull --ff-only origin main
  git switch -c feat/short-description
Use feat/, fix/, docs/, test/, refactor/ or chore/ prefixes.
Commit small units with informative messages such as:
  feat: add bookmark persistence
  fix: handle failed API requests
  test: cover empty cart state

## 4. Quality gates
Run dart format ., flutter analyze, and flutter test.
Build on a real Android phone for UI-facing changes.
Write a test for new business logic where practical.
Update README and screenshots when behavior changes.

## 5. Open a pull request
Push with git push -u origin YOUR_BRANCH_NAME.
Open a PR to upstream main. Link an issue using Closes #ISSUE_NUMBER.
```

```
Explain purpose, approach, manual tests and screenshots.
Request review from at least one team member.
Respond to review by pushing new commits to the same branch.
Do not merge failing checks. Use squash merge if the maintainer allows.

## 6. Conflicts and synchronization
With a fork, add upstream once:
  git remote add upstream git@github.com:COURSE_OWNER/csc303-mad.git
To synchronize:
  git fetch upstream
  git switch main
  git merge --ff-only upstream/main
Resolve conflicts on your feature branch, test again and push.
Do not force-push main; ask the maintainer before rewriting shared history.

## 7. Review checklist
[ ] Behavior matches the linked issue
[ ] Screenshots and README are updated
[ ] No secrets, generated builds or unrelated files
[ ] flutter analyze and flutter test pass
[ ] UI checked on an Android phone
[ ] Accessibility labels and empty/error states considered

## 8. Code of conduct
Provide specific and courteous technical feedback; credit contributors.
Plagiarism, copied submissions and fabricated results are prohibited.
The maintainer may request revisions before merging.
```

.github/PULL_REQUEST_TEMPLATE.md — create .github folder first

```
## Summary
Describe what changed and why.

## Related issue
Closes #ISSUE_NUMBER

## Validation
- [ ] dart format .
- [ ] flutter analyze
- [ ] flutter test
- [ ] Tested on a physical Android phone

## Screenshots / screen recording
Before / after:

## Risks and rollback
State network, database migration or compatibility risks.

## Reviewer notes
Mention difficult decisions, edge cases and files to inspect.
```

Contribution workflow: maintainer vs student

Repository maintainer. Create issues with clear acceptance criteria, label feature/bug/docs/test work, invite team members (or allow forks), protect main through branch rules when available, and merge only reviewed pull requests with checks.
Contributor. Claim an issue, branch from current main, implement one focused change, run dart format . / flutter analyze / flutter test, push the branch and open a PR with evidence.
Reviewer. Review correctness, usability, security and tests. Request changes when needed, and merge only after comments are resolved.

REVIEWED FEATURE BRANCH — execute from an existing Flutter repository

```
# Student with direct write access:
git switch main
git pull --ff-only origin main
git switch -c feat/contact-screen
# Edit your files, then run checks:
dart format .
flutter analyze
flutter test
```

```
git status
git add lib test README.md
git commit -m "feat: create contact screen"
git push -u origin feat/contact-screen
# Open a Pull Request on GitHub into main and request review.

# Optional: after merge, clean up the local feature branch:
git switch main
git pull --ff-only origin main
git branch -d feat/contact-screen
```

Conflict resolution exercise: two students change the same title in lib/main.dart. Create a second branch, make a different edit, merge, inspect <<<<<<< / ======= / >>>>>>> conflict markers, choose a correct single Dart expression, run flutter analyze, stage the resolved file and commit the merge. Do not push unresolved markers.

Scenario-based practice — unsolved

Scenario 1. Campus team repository. A three-person team must add a timetable feature and a UI bug fix without breaking main. Decide issue ownership, branch names, commit granularity, reviewer order and the evidence required for each PR. Deliver two issue links and two reviewed PRs.

Scenario 2. No device appears. A classmate installed Android Studio and Flutter but flutter devices reports no Android phone, while adb devices says unauthorized. Diagnose the exact permission, driver and toolchain steps; record commands/output and explain why an emulator image is not required.

Technical viva — justify your reasoning

1. Why is a commit not the same as a push?
2. What security property does a passphrase add to an SSH private key?
3. Why must only id_ed25519.pub be added to GitHub?
4. How does Android SDK Platform-Tools differ from Build-Tools?
5. Why does GitHub ssh -T sometimes exit with status 1 even after successful authentication?
6. What evidence tells you a USB connection is blocked at the ADB authorization stage?
7. How do a fork, remote origin and remote upstream differ?
8. Why should a team avoid force-pushing its protected main branch?

Submission and self-check

Submit screenshots of flutter doctor -v (hide personal paths if desired), adb devices, phone app, SSH authentication greeting, two commits and a reviewed PR; include CONTRIBUTING.md and the PR template. Never include your private key.

CSC303 / NOVEMBER 2026 REVISION

LAB 01 EXTENSION | Contribution workflow in the portfolio

The Lab 01 csc303-mad example continues to teach Git through its own starter project. From Lab 02 onward use the portfolio structure on the preceding page, or an instructor-provided course organization repository. Do not paste Flutter apps into the same project root as one another.

Repository owner versus contributor

Owner / maintainer	Student contributor
Creates portfolio, invites teammates; enables available branch rules and review settings	Clones repository; claims an issue; creates a feature branch
Reviews PRs and confirms required checks	Runs format, analyze, relevant tests and real-phone demonstration
Merges accepted PRs and manages release tag	Commits focused code; opens PR; responds to reviewer feedback

Repository protection controls depend on repository ownership and GitHub plan/settings. If protection cannot be configured in a particular classroom repository, require peer-reviewed pull requests by assessment policy and report the limitation instead of claiming protection is enabled.

Example next-lab contribution

```
# From csc303-portfolio/ after its main branch is pushed:
git switch main
git pull --ff-only origin main
git switch -c feat/lab02-dart
cd labs
flutter create --platforms=android mad_lab02
cd mad_lab02
# Edit source; run format, analyze and current tests.
dart format lib test
flutter analyze
cd ../..
git add labs/mad_lab02
git commit -m "feat: submit Lab 02 Dart app"
git push -u origin feat/lab02-dart
# Open PR; request a classmate review.
```

Acceptance evidence

- Show two meaningful commits, one successful pull request review, and the relevant Flutter project files in the PR diff.
- Show a real device recognized by ADB and a Flutter application launched on that device.
- Explain the role of origin/upstream; demonstrate that the private SSH key never leaves the device.
- Document whether the repository uses enforced branch protection or a classroom review-only procedure.

LAB 02 Dart foundations and Flutter project anatomy

Unit 1 • 3-hour guided laboratory • CLO-5

Learning goal. Apply Dart types, null safety, collections and reusable functions inside a Flutter interface and recognize the purpose of common project files.

Core concepts. Dart var/final/const; types and null safety; lists, map, where; functions; pubspec.yaml; lib/, android/, test/, assets; hot reload.

Before you start. Lab 01 toolchain and a recognized Android phone.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab02
cd mad_lab02
flutter run
```

Anatomy of a generated project

A Flutter Android application has one PROJECT ROOT: the folder containing pubspec.yaml. In Android Studio, use the Project or Project Files view (not the filtered Android view) to study the complete layout below. The assets/ directory is student-created; generated directories and exact native filenames may vary by Flutter version.

ILLUSTRATED DIRECTORY TREE | Example for flutter create --platforms=android mad_lab02

```
mad_lab02/                          <- PROJECT ROOT
|-- lib/
|   `-- main.dart                  <- start editing Dart UI here
|-- test/
|   `-- widget_test.dart           <- generated starter test
|-- android/
|   |-- app/
|   |   |-- src/main/
|   |   |   |-- AndroidManifest.xml
|   |   |   `-- kotlin/.../MainActivity.kt  <- native bridge
|   |   `-- build.gradle.kts        <- app Android configuration
|   |-- settings.gradle.kts        <- generated Gradle settings
|   `-- gradle.properties
|-- assets/                         <- create yourself, if needed
|   `-- notes.txt
|-- pubspec.yaml                    <- package metadata + assets
|-- pubspec.lock                    <- resolved dependencies
|-- analysis_options.yaml          <- Dart analysis rules
|-- .gitignore                      <- generated exclusions
|-- README.md                       <- project setup notes
|-- .dart_tool/                     <- generated; do not commit
`-- build/                          <- generated; do not commit
```

Read the tree from top to bottom: your Dart screen code belongs in lib/, Android-specific settings and native code belong in android/, automated tests belong in test/, and dependencies/assets are declared in pubspec.yaml. The tree omits several Gradle wrapper/support files to stay readable. A regular multi-platform project may also contain ios/, web/, macos/, linux/ and windows/; this course uses Android-only generation.

Path / file	Purpose and edit policy
pubspec.yaml	Single package manifest; configure dependencies, SDK constraints, version, and bundled assets. EDIT deliberately; YAML indentation matters.
pubspec.lock	Records resolved package versions for repeatable builds; for application projects normally keep it in source control.
lib/main.dart	Dart entry point; main() calls runApp() to attach the root widget. EDIT for each solved task.
lib/screens/, lib/widgets/	Optional folders STUDENTS create as an app grows; screens group pages and widgets hold reusable UI.
test/widget_test.dart	Flutter widget tests. Replace the generated starter test when your main.dart no longer contains its starter MyApp.

Path / file	Purpose and edit policy
android/app/src/main/AndroidManifest.xml	Android application label, activities and requested Android permissions; change permissions only when a feature requires them.
android/app/build.gradle.kts	Android application ID, SDK compatibility and build configuration. Some older templates use build.gradle (Groovy).
android/app/src/main/kotlin/.../MainActivity.kt	Android entry activity and optional platform-channel Kotlin code. Actual package directories match the generated organization ID.
android/settings.gradle.kts and Gradle wrapper	Android build infrastructure generated by Flutter; normally leave unchanged for introductory labs.
assets/	You create this directory for images, fonts or JSON; declare each asset (or appropriate directory) under flutter: in pubspec.yaml.
analysis_options.yaml	Static-analysis lint rules for Dart; flutter analyze uses these rules.
.gitignore	Excludes temporary output, local IDE state and secrets; preserve and extend the generated file rather than overwriting.
build/ and .dart_tool/	Generated build output and tool caches; do not modify or commit to Git.
README.md	Plain-language project purpose, prerequisites, setup commands, screenshots and contribution link.

How a Flutter application starts and displays a screen

EXECUTION FLOW | What happens when you press Run

```
Android launches the Flutter Activity
        |
Dart main() executes in lib/main.dart
        |
runApp(const RootWidget()) installs the widget tree
        |
MaterialApp selects theme, routes and home widget
        |
Scaffold arranges AppBar, body and floating actions
        |
Widgets are built, laid out and painted on the phone
```

The tree is declarative: widgets describe the interface at a moment in time, while Flutter decides how to lay out and repaint it. setState marks relevant State objects dirty and schedules new widget builds; it is not a direct instruction to paint pixels. Native MainActivity hosts the Flutter engine on Android but typically contains no Dart UI code.

Open, edit and run the correct files

WORKFLOW 1 | Open the project root

Choose File > Open and select mad_lab02 itself (the folder containing pubspec.yaml). Wait for package resolution. The preferred editor view for studying folders is Project / Project Files; the Android view is better for native Gradle inspection.

WORKFLOW 2 | Replace only the requested source file

When a solved activity is labelled lib/main.dart, select that file and replace its ENTIRE contents with the provided complete example. Do not paste the example into AndroidManifest.xml, MainActivity.kt or the build script. Save before running.

WORKFLOW 3 | Install packages only when instructed

flutter pub get resolves packages already declared in pubspec.yaml; flutter pub add PACKAGE_NAME adds a new dependency and fetches it. Most early labs use only the Flutter SDK and need no extra packages. Do not edit pubspec.lock manually.

REPRODUCIBLE VALIDATION | Use the project root; run commands in order

```
flutter pub get
dart format lib test
flutter analyze
flutter devices
flutter run
```

After the code: pub get downloads dependencies, dart format applies Dart formatting, analyze flags static errors, devices confirms the phone target, and run compiles, installs and starts the app. These checks should be executed inside the directory containing pubspec.yaml. If your generated starter widget test still references the old app after you replace main.dart, first update that test to match the new class before running flutter test.

How pubspec.yaml controls packages and assets

This YAML file is configuration, not Dart code. The dependencies section lists runtime packages; dev_dependencies lists tools such as flutter_test and linters. The flutter: section configures Flutter-specific assets and fonts. Keep indentation with spaces; tabs can break YAML. The small example below is an EDIT TO THE EXISTING flutter: section, not a complete replacement pubspec.yaml.

ADD A LOCAL TEXT ASSET | Git Bash / Linux / macOS terminal

```
mkdir -p assets
printf 'CSC303: bundled local data\n' > assets/notes.txt
```

EDIT EXISTING pubspec.yaml | Insert assets: under the existing flutter: key

```
flutter:
  uses-material-design: true
  assets:
    - assets/notes.txt
```

Explanation: create the file BEFORE declaring it; the asset path is relative to pubspec.yaml. Do not add a SECOND flutter: block; insert assets: into the block Flutter generated, then save and run flutter pub get. If the example app does not load this asset yet, its inclusion changes the bundle but not the screen. Official guidance: https://docs.flutter.dev/ui/assets/assets-and-images.

AndroidManifest.xml, Gradle and generated Kotlin code

AndroidManifest.xml is the native Android declaration of app components and permissions. For an HTTP/HTTPS API feature, release builds commonly need the INTERNET permission. Put the one-line declaration directly inside the existing manifest root ONLY when networking is needed; never replace the entire manifest with this line.

ANDROID PERMISSION FRAGMENT | Add inside the existing <manifest> element if needed

```
<uses-permission android:name="android.permission.INTERNET" />
```

The Android app Gradle file (usually android/app/build.gradle.kts in modern templates) controls namespace, applicationId, SDK levels and release signing. applicationId identifies the installed app; changing it can affect Firebase registrations or updates. compileSdk is the Android API used to compile, minSdk is the oldest supported Android version, and targetSdk communicates intended Android behavior. Flutter templates supply compatible defaults: edit only when a package, device policy or deployment requirement actually calls for it.

MainActivity.kt is a native Android entry point, normally a short class that extends FlutterActivity. Dart UI changes belong in lib/, not MainActivity. Lab 14 later adds a method channel as an intentional exception. Do not commit android/local.properties (local SDK paths), generated build output, or signing credentials.

Hot reload, hot restart and debugging a real phone

DEBUG 1 | Make a Dart-only UI change

While flutter run is active, modify a Text label in lib/main.dart and save. Press r in the terminal or use Android Studio Hot Reload. Hot reload usually preserves the current widget state, so counters and form inputs often remain unchanged.

DEBUG 2 | Know when to restart

Press R in the flutter run terminal for Hot Restart: the Dart app is restarted and local widget state is reset. If you change AndroidManifest.xml, native Kotlin, Gradle, permissions, plugins or native resources, stop the process and start a new build; hot reload cannot apply native Android code changes.

DEBUG 3 | Debug logically rather than guessing

Click in the left gutter beside a Dart line to set a breakpoint, start Debug, trigger the event on the phone, then inspect locals and the call stack. Use Run/Debug output for Dart errors, the Flutter Inspector for widget-tree/layout problems, and Android Logcat for native crashes. A yellow-black overflow warning is a layout problem, not a compiler error.

FINAL LAB 02 CHECKS | Run only with a widget test matching the CURRENT app

```
dart format lib test
flutter analyze
flutter test
flutter build apk --debug
```

IMPORTANT: flutter create includes a starter widget test tied to the generated starter app. Replacing lib/main.dart for either solved task invalidates that test unless you rewrite it. Use the working example in Lab 01 as a test replacement pattern. The debug APK is written under build/app/outputs/flutter-apk/. Official guidance: https://docs.flutter.dev/tools/hot-reload and https://docs.flutter.dev/tools/android-studio.

Solved task 1 | Reusable Dart function: grade analyzer

Problem and objective. Create a grading widget that converts assignment, midterm and final raw marks into a 100-point score and displays a grade. This is a PROGRAMMING exercise and does not replace the official course grading policy.

lib/main.dart

```
import 'package:flutter/material.dart';

String resultFor(double a, double m, double f) {
  final score = a + m + f;
  if (score >= 85) return 'A';
  if (score >= 70) return 'B';
  if (score >= 60) return 'C';
  if (score >= 50) return 'D';
  return 'F';
}

void main() => runApp(const MaterialApp(home: GradeDemo()));

class GradeDemo extends StatefulWidget {
  const GradeDemo({super.key});
  @override
  State<GradeDemo> createState() => _GradeDemoState();
}

class _GradeDemoState extends State<GradeDemo> {
  double assignment = 20, midterm = 18, finalExam = 42;

  @override
  Widget build(BuildContext context) {
    final score = assignment + midterm + finalExam;
    return Scaffold(
      appBar: AppBar(title: const Text('Dart grade analyzer')),
      body: ListView(padding: const EdgeInsets.all(18), children: [
        for (final item in [
          ('Assignment /25', assignment, 25.0),
          ('Midterm /25', midterm, 25.0),
          ('Final /50', finalExam, 50.0),
        ])
          Column(children: [
            Text('${item.$1}: ${item.$2.toStringAsFixed(0)}'),
            Slider(
              value: item.$2, max: item.$3,
              onChanged: (v) => setState(() {
                if (item.$1.startsWith('Assignment')) assignment = v;
                if (item.$1.startsWith('Midterm')) midterm = v;
                if (item.$1.startsWith('Final')) finalExam = v;
              }),
            ),
          ]),
        Card(child: Padding(
          padding: const EdgeInsets.all(20),
          child: Text('Total: ${score.toStringAsFixed(0)}/100  '
              'Grade: ${resultFor(assignment, midterm, finalExam)}'),
        )),
      ]),
```

```
    );
  }
}
```

How the solution works. The three numeric state fields store user inputs. A pure function resultFor maps the sum into a grade category. Record fields item.$1, item.$2 and item.$3 demonstrate positional records in modern Dart; Slider clamps input to its declared maximum.

Expected output. Moving sliders updates totals and the grade immediately. The example starts at 80/100 (B).

Check your work. Try 25+25+50 and 0+0+0, then explain why the result function is simpler to test than calculations embedded in build().

Solved task 2 | Lists, maps and null safety in a directory

Problem and objective. Use in-memory structured data, filtering and optional values in a student directory. The example does not use a server.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: DirectoryPage()));

class Student {
  final String name;
  final String department;
  final String? phone;
  const Student(this.name, this.department, this.phone);
}

class DirectoryPage extends StatefulWidget {
  const DirectoryPage({super.key});
  @override
  State<DirectoryPage> createState() => _DirectoryPageState();
}

class _DirectoryPageState extends State<DirectoryPage> {
  final entries = const [
    Student('Ayesha', 'CS', '0300-0000001'),
    Student('Hamza', 'SE', null),
    Student('Hina', 'CS', '0300-0000003'),
    Student('Bilal', 'SE', null),
  ];
  String selected = 'All';

  @override
  Widget build(BuildContext context) {
    final visible = entries.where((s) =>
        selected == 'All' || s.department == selected).toList();
    return Scaffold(
      appBar: AppBar(title: const Text('Student directory')),
      body: Column(children: [
        SegmentedButton<String>(
          segments: const [
            ButtonSegment(value: 'All', label: Text('All')),
            ButtonSegment(value: 'CS', label: Text('CS')),
            ButtonSegment(value: 'SE', label: Text('SE')),
          ],
          selected: {selected},
          onSelectionChanged: (v) => setState(() => selected = v.first),
        ),
        Expanded(child: ListView.builder(
          itemCount: visible.length,
          itemBuilder: (context, i) {
            final s = visible[i];
            return ListTile(
              leading: CircleAvatar(child: Text(s.name[0])),
              title: Text(s.name),
              subtitle: Text('${s.department} • ${s.phone ?? 'No phone'}'),
            );
```

```
          },
        )),
      ]),
    );
  }
}
```

How the solution works. Student is an immutable data model. Its phone is nullable; the ?? operator supplies safe fallback text. List.where returns the selected subset and ListView.builder renders only needed list children.

Expected output. All displays four people; CS and SE each display two. Missing numbers show No phone.

Check your work. Add one entry with no phone, change the selected filter and confirm no null exception occurs.

Scenario-based practice — unsolved

Scenario 1. Merit calculation. A scholarship form must compare GPAs and reject invalid negative or greater-than-4 values. Specify types, validation rules, test boundaries and use a pure function.

Scenario 2. Class roster. The CS office must group records by semester and search partial names while gracefully handling missing emails. Design a Student model and user-facing error states.

Technical viva — justify your reasoning

1. Why can const objects improve allocations and rebuild cost?
2. When do final and const differ?
3. Why is a nullable String? not assignable directly to String?
4. What does List.where return before toList() is called?
5. Why should grading thresholds be tested at exact boundaries?
6. Which changes require hot restart instead of hot reload?
7. Why can an Android release app fail to call an API even when the debug app connects successfully? Which file would you inspect?
8. Distinguish pubspec.yaml from pubspec.lock, and explain which files should be committed for an application project.

Submission and self-check

Both screenshots, flutter analyze output, commit SHA and a short explanation of null safety and project structure.

LAB 03 Widget tree, layouts and adaptive UI

Unit 2 • 3-hour guided laboratory • CLO-5

Learning goal. Compose readable screens using Row, Column, Expanded, Stack and LayoutBuilder without common overflow errors.

Core concepts. StatelessWidget, widget composition, constraints, responsive breakpoints, Card, SafeArea, MediaQuery, Flutter image/icon resources.

Before you start. Create two new copies of a Lab 03 project so each code example can replace lib/main.dart.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab03
cd mad_lab03
```

Solved task 1 | Responsive faculty profile page

Problem and objective. Build a reusable profile card that places portrait and detail side by side on wide screens but stacks content on narrow phones.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: ProfilePage()));

class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Faculty profile')),
      body: SafeArea(child: LayoutBuilder(builder: (context, size) {
        final wide = size.maxWidth >= 520;
        final portrait = Container(
          width: wide ? 140 : 100, height: 120,
          decoration: BoxDecoration(
            color: Colors.teal.shade100,
            borderRadius: BorderRadius.circular(16),
          ),
          child: const Icon(Icons.person, size: 75),
        );
        final details = const Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Muhammad Jamshaid Iqbal',
                style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
            SizedBox(height: 8),
            Text('Department of Computer Science'),
            Text('COMSATS University Islamabad – Sahiwal'),
            SizedBox(height: 8),
            Chip(label: Text('Mobile Application Development')),
          ],
        );
        return SingleChildScrollView(
          padding: const EdgeInsets.all(20),
          child: Card(child: Padding(
            padding: const EdgeInsets.all(16),
            child: wide
                ? Row(children: [portrait, const SizedBox(width: 18),
                    Expanded(child: details)])
                : Column(children: [portrait, const SizedBox(height: 16),
                    details]),
          )),
        );
      })),
```

```
    );
  }
}
```

How the solution works. LayoutBuilder exposes available width from the parent. The breakpoint changes the child arrangement, Expanded prevents the details from exceeding the horizontal Row, and SingleChildScrollView protects against small screens and large accessibility text.

Expected output. A narrow Android phone shows the icon above the profile; a tablet or landscape width may show a side-by-side card.

Check your work. Rotate the device; test with larger system font size and verify that no black/yellow overflow stripes appear.

Solved task 2 | Responsive product dashboard with adaptive grid

Problem and objective. Show mock inventory cards with different grid density depending on available screen width.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: InventoryPage()));

class InventoryPage extends StatelessWidget {
  const InventoryPage({super.key});
  static const items = [
    ('Laptop', Icons.laptop, 129000),
    ('Mouse', Icons.mouse, 2200),
    ('Keyboard', Icons.keyboard, 4900),
    ('Phone', Icons.smartphone, 68000),
    ('Headphones', Icons.headphones, 5500),
    ('Tablet', Icons.tablet, 87000),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Responsive inventory')),
      body: LayoutBuilder(builder: (context, size) {
        final columns = size.maxWidth >= 700 ? 4 :
            size.maxWidth >= 420 ? 3 : 2;
        return GridView.builder(
          padding: const EdgeInsets.all(12),
          gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: columns, crossAxisSpacing: 12,
            mainAxisSpacing: 12, childAspectRatio: .90,
          ),
          itemCount: items.length,
          itemBuilder: (context, i) {
            final item = items[i];
            return Card(child: Padding(
              padding: const EdgeInsets.all(8),
              child: Column(mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  Icon(item.$2, size: 38, color: Colors.teal),
                  const SizedBox(height: 8),
                  Text(item.$1, textAlign: TextAlign.center),
                  Text('Rs. ${item.$3}', textAlign: TextAlign.center),
                ],
              ),
            ));
          },
        );
      }),
    );
  }
}
```

How the solution works. The grid delegate computes cell sizes from count and aspect ratio. Two, three or four columns are chosen by container width rather than a fixed screen assumption; builder avoids constructing off-screen cells unnecessarily.

Expected output. Six cards render in an adaptive two-to-four-column grid without requiring image downloads.

Check your work. Check portrait and landscape orientation and explain which threshold gets selected at 420 and 700 pixels.

Scenario-based practice — unsolved

Scenario 1. Accessible noticeboard. Create a noticeboard that remains usable with 200% font scaling and narrow screens. Specify scroll strategy, minimum touch target and semantic labels.

Scenario 2. Cafeteria product cards. Design a price card that switches layouts between portrait and landscape while preserving content order and accessible labels.

Technical viva — justify your reasoning

1. What exactly are box constraints and why can Row overflow?
2. When should Expanded be used instead of Flexible?
3. Why is LayoutBuilder often more precise than reading global screen width?
4. How does ListView.builder avoid eager work?
5. Does a Stack automatically become scrollable? Explain.
6. What testing proves a layout is accessible for text scaling?

Submission and self-check

Capture phone portrait and landscape, record chosen grid counts and show flutter analyze without errors.

CSC303 / NOVEMBER 2026 REVISION

LAB 03 EXTENSION | Repeatable accessibility checks

Responsive layout is not only avoiding a RenderFlex overflow at the default font size. Students should verify that the interface remains usable when text is enlarged, when screen readers are enabled and when orientation changes.

Mandatory manual checks

Test	How to perform it	Pass condition
Large text	Raise device font/display size; repeat at an enlarged scale	Text can be read; actions remain available; no clipping
Portrait and landscape	Rotate phone; test the narrowest supported viewport	No overflow; navigation and buttons remain reachable
Screen-reader labels	Enable TalkBack; navigate without visual targeting	Actionable controls have meaningful spoken labels
Touch interaction	Try controls with one hand and on a small phone	Interactive targets are easily distinguishable and operable
Theme and contrast	Inspect light/dark themes and error states	Meaning conveyed by more than color alone

Example Semantics for an icon-only action

```
IconButton(
  tooltip: 'Delete saved note',
  icon: const Icon(Icons.delete_outline),
  onPressed: deleteNote,
)
```

Prefer meaningful built-in labels and tooltips; do not wrap every widget in duplicate Semantics nodes. Use the Accessibility Inspector or device screen reader for verification. For animations, check reduced-motion behavior in Lab 14.

Student exercise

- In Lab 03 Task 1, demonstrate narrow and wide layouts and capture screen images at normal and enlarged text.
- Record one actual accessibility failure and the source-code change that resolves it.
- Attach the relevant before/after evidence to the lab repository; avoid including any private device data.

LAB 04 Input forms, validation and local UI state

Unit 2 • 3-hour guided laboratory • CLO-5

Learning goal. Build interactive forms using TextFormField, selection widgets, a Form validator, controllers and setState.

Core concepts. FormState, GlobalKey, TextEditingController disposal, CheckboxListTile, RadioGroup alternatives, SwitchListTile, dialogs and SnackBar feedback.

Before you start. Lab 03 UI knowledge; use an independent new project or replace lib/main.dart between tasks.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab04
cd mad_lab04
```

Solved task 1 | Validated lab registration form

Problem and objective. A student must supply name, registration number, select a lab section and accept an academic-integrity declaration before submission.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: RegisterPage()));

class RegisterPage extends StatefulWidget {
  const RegisterPage({super.key});
  @override
  State<RegisterPage> createState() => _RegisterPageState();
}

class _RegisterPageState extends State<RegisterPage> {
  final key = GlobalKey<FormState>();
  final name = TextEditingController();
  final roll = TextEditingController();
  String section = 'A';
  bool agreed = false;

  @override
  void dispose() {
    name.dispose(); roll.dispose(); super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Register for MAD lab')),
      body: Form(key: key, child: ListView(
        padding: const EdgeInsets.all(20), children: [
          TextFormField(
            controller: name,
            decoration: const InputDecoration(labelText: 'Full name'),
            validator: (s) => (s == null || s.trim().length < 3)
                ? 'Enter at least 3 characters' : null,
          ),
          TextFormField(
            controller: roll,
            decoration: const InputDecoration(labelText: 'Roll number'),
            validator: (s) => (s == null || s.trim().isEmpty)
                ? 'Roll number is required' : null,
          ),
          DropdownButtonFormField<String>(
            initialValue: section,
            items: const [
              DropdownMenuItem(value: 'A', child: Text('Section A')),
              DropdownMenuItem(value: 'B', child: Text('Section B')),
            ],
            onChanged: (v) => setState(() => section = v ?? 'A'),
```

```
          ),
          CheckboxListTile(
            value: agreed,
            title: const Text('I will submit original work'),
            onChanged: (v) => setState(() => agreed = v ?? false),
          ),
          FilledButton(
            onPressed: () {
              if (!key.currentState!.validate()) return;
              if (!agreed) {
                ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
                  content: Text('Accept the declaration first')));
                return;
              }
              showDialog<void>(context: context, builder: (context) =>
                AlertDialog(
                  title: const Text('Registered'),
                  content: Text('${name.text.trim()} (${roll.text.trim()}) '
                      'in Section $section'),
                  actions: [TextButton(onPressed: () => Navigator.pop(context),
                      child: const Text('OK'))],
                ));
            },
            child: const Text('Register'),
          ),
        ],
      )),
    );
  }
}
```

How the solution works. Validators return a message for invalid values and null for valid values. GlobalKey<FormState> invokes all attached field validators. The declaration is additional state checked after the Form is valid. Controllers are disposed to release listeners.

Expected output. Invalid submissions show field-specific messages; a complete registration shows a summary dialog.

Check your work. Test blank fields, two-character names, unchecked declaration and a valid submission; demonstrate each validation result.

Solved task 2 | Radio-like choice, switch and computed bill

Problem and objective. Create a meal selector using SegmentedButton as an accessible mutually-exclusive choice, a delivery switch and live calculated total.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MealPage()));

class MealPage extends StatefulWidget {
  const MealPage({super.key});
  @override
  State<MealPage> createState() => _MealPageState();
}

class _MealPageState extends State<MealPage> {
  String meal = 'Lunch';
  int quantity = 1;
  bool delivery = false;
  final prices = {'Breakfast': 200, 'Lunch': 350, 'Dinner': 450};

  @override
  Widget build(BuildContext context) {
    final total = prices[meal]! * quantity + (delivery ? 100 : 0);
    return Scaffold(
      appBar: AppBar(title: const Text('Meal estimator')),
      body: ListView(padding: const EdgeInsets.all(16), children: [
        SegmentedButton<String>(
          segments: const [
```

```
            ButtonSegment(value: 'Breakfast', label: Text('Breakfast')),
            ButtonSegment(value: 'Lunch', label: Text('Lunch')),
            ButtonSegment(value: 'Dinner', label: Text('Dinner')),
          ],
          selected: {meal},
          onSelectionChanged: (v) => setState(() => meal = v.first),
        ),
        Row(children: [
          const Expanded(child: Text('Quantity')),
          IconButton(onPressed: quantity > 1
              ? () => setState(() => quantity--) : null,
              icon: const Icon(Icons.remove)),
          Text('$quantity'),
          IconButton(onPressed: () => setState(() => quantity++),
              icon: const Icon(Icons.add)),
        ]),
        SwitchListTile(
          title: const Text('Home delivery (Rs. 100)'),
          value: delivery,
          onChanged: (v) => setState(() => delivery = v),
        ),
        const Divider(),
        Text('Total: Rs. $total',
            style: Theme.of(context).textTheme.headlineMedium),
      ]),
    );
  }
}
```

How the solution works. The meal selection acts like a group of radio buttons: exactly one selection is active. The total is derived in build from the current state, so it cannot go stale. The disabled decrement button enforces a minimum quantity of one.

Expected output. Meal choice, quantity buttons and delivery switch immediately modify the estimated total.

Check your work. Verify the quantity never falls below 1 and derive the expected price for Dinner × 2 + delivery.

Scenario-based practice — unsolved

Scenario 1. Course feedback. Design a feedback form requiring a rating, bounded comment length and anonymous opt-in. Specify validation, accessible controls and safe reset behavior.

Scenario 2. Admission fee. A student selects program, transport and hostel needs; compute fees but ensure no negative quantity and display clear error messages.

Technical viva — justify your reasoning

1. Why does calling setState outside State produce an architectural error?
2. Why dispose TextEditingController?
3. Why is a validator returning null interpreted as successful validation?
4. What is the danger of storing a computed total in a separate mutable field?
5. When is SnackBar more appropriate than AlertDialog?
6. Which state should survive an app restart, and would setState alone achieve that?

Submission and self-check

Submit valid and invalid screenshots, the exact input test cases and a Git commit labeled feat: validated lab form.

LAB 05 Navigation, routes and passing data between screens

Unit 2 • 3-hour guided laboratory • CLO-5

Learning goal. Implement forward navigation, detail screens, reverse navigation with values and named routes.

Core concepts. Navigator stack (push/pop), route arguments, MaterialPageRoute, mounted/context safety, user-flow planning.

Before you start. Students know StatelessWidget and StatefulWidget; every task is an independent app.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab05
cd mad_lab05
```

Solved task 1 | Course list → course detail → saved selection

Problem and objective. Tap a course to navigate to its details, then return the selected course name as a value to the original screen.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: CoursesPage()));

class CoursesPage extends StatefulWidget {
  const CoursesPage({super.key});
  @override
  State<CoursesPage> createState() => _CoursesPageState();
}

class _CoursesPageState extends State<CoursesPage> {
  String selected = 'No course selected';
  final courses = const ['Mobile Application Development',
    'Database Systems', 'Software Engineering'];

  Future<void> openDetail(String course) async {
    final result = await Navigator.push<String>(context,
      MaterialPageRoute(builder: (_) => DetailPage(name: course)));
    if (!mounted || result == null) return;
    setState(() => selected = result);
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Course navigator')),
    body: Column(children: [
      Padding(padding: const EdgeInsets.all(16),
        child: Text('Chosen: $selected')),
      Expanded(child: ListView(
        children: courses.map((name) => ListTile(
          title: Text(name), trailing: const Icon(Icons.chevron_right),
          onTap: () => openDetail(name),
        )).toList(),
      )),
    ]),
  );
}

class DetailPage extends StatelessWidget {
  final String name;
  const DetailPage({super.key, required this.name});

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Course details')),
    body: Center(child: Column(mainAxisSize: MainAxisSize.min,
      children: [
```

```
        Text(name, style: Theme.of(context).textTheme.titleLarge),
        const SizedBox(height: 18),
        FilledButton(
          onPressed: () => Navigator.pop(context, name),
          child: const Text('Choose course'),
        ),
      ],
    )),
  );
}
```

How the solution works. push places a detail route on the navigation stack and yields a Future. pop(context, name) removes that route and resolves the Future with the chosen item. The mounted check prevents state changes after the original page has been disposed.

Expected output. Selecting a course opens a new page; pressing Choose course returns and updates the first page.

Check your work. Press Android Back without choosing and explain why the original selection remains unchanged.

Solved task 2 | Named routes with typed arguments

Problem and objective. Open a profile screen via a named route and pass a simple profile title in RouteSettings.arguments.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(MaterialApp(
  initialRoute: '/',
  onGenerateRoute: (settings) {
    if (settings.name == '/') {
      return MaterialPageRoute(builder: (_) => const HomePage());
    }
    if (settings.name == '/profile') {
      final label = settings.arguments as String? ?? 'Guest';
      return MaterialPageRoute(
        builder: (_) => ProfilePage(label: label), settings: settings);
    }
    return MaterialPageRoute(builder: (_) => const ErrorPage());
  },
));

class HomePage extends StatelessWidget {
  const HomePage({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Named routes')),
    body: Center(child: FilledButton(
      onPressed: () => Navigator.pushNamed(context, '/profile',
          arguments: 'Student representative'),
      child: const Text('Open profile'),
    )),
  );
}

class ProfilePage extends StatelessWidget {
  final String label;
  const ProfilePage({super.key, required this.label});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Profile')),
    body: Center(child: Text(label)),
  );
}

class ErrorPage extends StatelessWidget {
  const ErrorPage({super.key});
  @override
  Widget build(BuildContext context) => const Scaffold(
    body: Center(child: Text('Route not found')),
  );
}
```

How the solution works. onGenerateRoute centralizes name-to-page mapping and reads RouteSettings.arguments. The fallback route avoids silent failure for an unknown path. Type casting is guarded with as String? for null, but a wrong non-String type would still throw; typed route objects scale better in large apps.

Expected output. The Home screen navigates to a profile labeled Student representative; the app bar back arrow returns home.

Check your work. Change /profile to /missing and confirm the fallback screen renders.

Scenario-based practice — unsolved

Scenario 1. Exam workflow. Model a three-screen exam flow: instructions, timed questions and results. Decide when the back button is allowed and how an incomplete attempt should be handled.

Scenario 2. Order details. Navigate from orders to an order detail and back with an edited delivery note; define type-safe data transfer and cancellation behavior.

Technical viva — justify your reasoning

1. What is the difference between pushReplacement and push?
2. How does Navigator.pop return a typed value?
3. Why does a Future from push complete after the route is popped?
4. What happens to state if a widget is unmounted during an awaited navigation?
5. What risk follows casting route arguments without validation?
6. How would deep links change your route architecture?

Submission and self-check

Demonstrate push, pop, cancellation and unknown-route behavior; attach a two-screen navigation diagram.

LAB 06 Scrollable lists, cards, search and user feedback

Unit 3 • 3-hour guided laboratory • CLO-5

Learning goal. Render large item collections efficiently, filter results and manage deletion through dialogs and undo patterns.

Core concepts. ListView.builder, Card, ListTile, Dismissible, confirm dialogs, SnackBarAction, controller cleanup, stable keys.

Before you start. Create a separate project for each solved task or replace lib/main.dart between executions.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab06
cd mad_lab06
```

Solved task 1 | Searchable noticeboard with lazy list items

Problem and objective. Simulate 200 announcements and filter them by their title. Tapping a tile should show the selected announcement in a SnackBar.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: NoticePage()));

class NoticePage extends StatefulWidget {
  const NoticePage({super.key});
  @override
  State<NoticePage> createState() => _NoticePageState();
}

class _NoticePageState extends State<NoticePage> {
  final search = TextEditingController();
  final notices = List.generate(200, (i) => 'Campus notice ${i + 1}');
  @override
  void dispose() { search.dispose(); super.dispose(); }

  @override
  Widget build(BuildContext context) {
    final visible = notices.where((e) =>
      e.toLowerCase().contains(search.text.toLowerCase())).toList();
    return Scaffold(
      appBar: AppBar(title: const Text('Campus notices')),
      body: Column(children: [
        Padding(padding: const EdgeInsets.all(12), child: TextField(
          controller: search,
          decoration: const InputDecoration(
            prefixIcon: Icon(Icons.search), labelText: 'Search notices',
            border: OutlineInputBorder(),
          ),
          onChanged: (_) => setState(() {}),
        )),
        Expanded(child: ListView.builder(
          itemCount: visible.length,
          itemBuilder: (context, i) => Card(
            child: ListTile(
              leading: const Icon(Icons.campaign_outlined),
              title: Text(visible[i]),
              subtitle: const Text('COMSATS Sahiwal'),
              onTap: () => ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(content: Text('Opened ${visible[i]}'))),
            ),
          ),
        )),
      ]),
    );
  }
}
```

How the solution works. List.generate provides sample records; filtering occurs each rebuild, sufficient for 200 values. ListView.builder creates visible rows on demand and only one scrollable remains in the available Expanded area.

Expected output. The student sees 200 entries, can search 18 or another number, and receives a SnackBar on tap.

Check your work. Search for 999; the list should show zero rows without an exception.

Solved task 2 | Dismissible task list with confirmation and undo

Problem and objective. Create a list where swipe-to-delete requests confirmation, then offers a reversible deletion.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: SwipePage()));

class SwipePage extends StatefulWidget {
  const SwipePage({super.key});
  @override
  State<SwipePage> createState() => _SwipePageState();
}

class _SwipePageState extends State<SwipePage> {
  final List<String> tasks = ['Read Flutter docs',
    'Finish layout lab', 'Review pull request', 'Run widget tests'];

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Swipe tasks')),
    body: ListView.builder(
      itemCount: tasks.length,
      itemBuilder: (context, i) {
        final item = tasks[i];
        return Dismissible(
          key: ValueKey(item),
          background: Container(color: Colors.red.shade100,
            alignment: Alignment.centerLeft,
            child: const Icon(Icons.delete_outline)),
          confirmDismiss: (_) async => await showDialog<bool>(
            context: context,
            builder: (dialogContext) => AlertDialog(
              title: const Text('Delete task?'),
              content: Text(item),
              actions: [
                TextButton(onPressed: () => Navigator.pop(dialogContext, false),
                    child: const Text('Cancel')),
                FilledButton(onPressed: () => Navigator.pop(dialogContext, true),
                    child: const Text('Delete')),
              ],
            ),
          ) ?? false,
          onDismissed: (_) {
            setState(() => tasks.remove(item));
            ScaffoldMessenger.of(context).hideCurrentSnackBar();
            ScaffoldMessenger.of(context).showSnackBar(SnackBar(
              content: Text('Deleted $item'),
              action: SnackBarAction(label: 'UNDO', onPressed: () {
                setState(() => tasks.insert((i < tasks.length ? i : tasks.length), item));
              }),
            ));
          },
          child: ListTile(title: Text(item)),
        );
      },
    ),
  );
}
```

How the solution works. Dismissible needs a stable identity key so Flutter can reconcile reordered or removed rows. confirmDismiss waits for user consent before moving the item. SnackBarAction inserts the removed value back into the list.

Expected output. Swiping produces a confirmation; Delete removes the row and Undo restores it.

Check your work. Dismiss the second item and tap Undo. Explain why list index alone is an unsafe key after deletion.

Scenario-based practice — unsolved

Scenario 1. Large library catalog. Build a 5,000-book catalog with search and category filters. Explain how you would avoid rebuilding every tile and debounce expensive searches.

Scenario 2. Student attendance. Allow swiping absent records with a confirmation and undo. Explain what server/database changes are required for a truly reliable undo.

Technical viva — justify your reasoning

1. Why does a Dismissible widget require a key?
2. What rebuild cost does ListView.builder save relative to Column with all children?
3. What data consistency issue arises if Undo is offered after a server delete?
4. When is an AlertDialog preferable to a SnackBar?
5. What happens to list state when the app process is killed?
6. How can stable unique IDs prevent state mixing between rows?

Submission and self-check

Provide search screenshots, successful undo demonstration, and an explanation of identity keys.

LAB 07 AppBar, Drawer, bottom navigation, tabs and themes

Unit 3 • 3-hour guided laboratory • CLO-5

Learning goal. Implement commonly used mobile navigation surfaces and verify their interaction and accessibility.

Core concepts. Scaffold drawer, NavigationBar, IndexedStack, DefaultTabController, TabBar, PopupMenuButton, Material 3 themes.

Before you start. This lab forms a sensible endpoint for the 25-mark midterm practical preparation.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab07
cd mad_lab07
```

Solved task 1 | Integrated Drawer and bottom navigation

Problem and objective. Navigate between Home, Courses and Account with a Material 3 bottom bar, and open a Drawer destination.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: CampusShell()));

class CampusShell extends StatefulWidget {
  const CampusShell({super.key});
  @override
  State<CampusShell> createState() => _CampusShellState();
}

class _CampusShellState extends State<CampusShell> {
  int page = 0;
  final titles = const ['Home', 'Courses', 'Account'];
  final views = const [
    Center(child: Text('Campus dashboard')),
    Center(child: Text('Enrolled courses')),
    Center(child: Text('Student account')),
  ];

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: Text(titles[page])),
    drawer: Drawer(child: SafeArea(child: ListView(children: [
      const UserAccountsDrawerHeader(
        accountName: Text('CSC303 Student'),
        accountEmail: Text('student@example.com')),
      ListTile(leading: const Icon(Icons.home),
        title: const Text('Home'), onTap: () {
          Navigator.pop(context); setState(() => page = 0);
        }),
      ListTile(leading: const Icon(Icons.person),
        title: const Text('Account'), onTap: () {
          Navigator.pop(context); setState(() => page = 2);
        }),
    ]))),
    body: IndexedStack(index: page, children: views),
    bottomNavigationBar: NavigationBar(
      selectedIndex: page,
      onDestinationSelected: (value) => setState(() => page = value),
      destinations: const [
        NavigationDestination(icon: Icon(Icons.home), label: 'Home'),
        NavigationDestination(icon: Icon(Icons.book), label: 'Courses'),
        NavigationDestination(icon: Icon(Icons.person), label: 'Account'),
      ],
    ),
  );
}
```

How the solution works. Scaffold owns the navigation drawer; close the drawer before updating the selected page. NavigationBar modifies an integer selection. IndexedStack retains each child subtree while changing which screen is visible.

Expected output. Bottom navigation and drawer links reach the appropriate label; the app bar title follows the selection.

Check your work. Change pages rapidly and verify bottom selection, drawer behavior and Android Back.

Solved task 2 | Tabbed course workspace with theme and action menu

Problem and objective. Provide Notes and Tasks tabs with a PopupMenuButton that toggles the global theme mode.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const ThemeTabsApp());

class ThemeTabsApp extends StatefulWidget {
  const ThemeTabsApp({super.key});
  @override
  State<ThemeTabsApp> createState() => _ThemeTabsAppState();
}

class _ThemeTabsAppState extends State<ThemeTabsApp> {
  bool dark = false;

  @override
  Widget build(BuildContext context) => MaterialApp(
    theme: ThemeData(colorSchemeSeed: Colors.indigo, useMaterial3: true),
    darkTheme: ThemeData(colorSchemeSeed: Colors.indigo,
      brightness: Brightness.dark, useMaterial3: true),
    themeMode: dark ? ThemeMode.dark : ThemeMode.light,
    home: DefaultTabController(length: 2, child: Scaffold(
      appBar: AppBar(
        title: const Text('Course workspace'),
        actions: [PopupMenuButton<String>(
          onSelected: (v) { if (v == 'theme') setState(() => dark = !dark); },
          itemBuilder: (_) => [PopupMenuItem(
            value: 'theme', child: Text(dark ? 'Light mode' : 'Dark mode'))],
        )],
        bottom: const TabBar(tabs: [
          Tab(icon: Icon(Icons.note), text: 'Notes'),
          Tab(icon: Icon(Icons.task), text: 'Tasks'),
        ]),
      ),
      body: const TabBarView(children: [
        Center(child: Text('Lecture notes appear here')),
        Center(child: Text('Pending lab exercises appear here')),
      ]),
    )),
  );
}
```

How the solution works. DefaultTabController coordinates TabBar and TabBarView; both must have the same number of tabs. ThemeMode switches material palettes at the root so descendant widgets inherit color changes. The popup represents infrequent global actions.

Expected output. Swipe horizontally to switch tabs; use the overflow menu to toggle dark and light appearance.

Check your work. Inspect readability and text contrast in both themes and show that tabs still navigate.

Midterm lab practical preparation

Suggested timed midterm (25 marks): build a two-screen course dashboard with validated search or entry, a responsive list of at least 15 records, working navigation, one dialog and a Git commit. Sample marking: core functionality 12, UI/accessibility 5, architecture/code quality 4, debugging/viva 4. Students should use flutter analyze before submitting.

Scenario-based practice — unsolved

Scenario 1. Three-area faculty app. Choose a correct mix of drawer, tabs and bottom navigation for Home, Timetable, Faculty and Settings. Justify which destinations are primary and which are secondary.

Scenario 2. Theme requirements. Design a night-mode reader with system setting support; explain how you would persist the chosen mode and avoid unreadable combinations.

Technical viva — justify your reasoning

1. Why is IndexedStack useful for bottom navigation state?
2. When is a Drawer a poor substitute for bottom navigation?
3. What invariant must hold between TabBar and TabBarView?
4. How does ThemeData differ from ThemeMode?
5. How can color psychology conflict with minimum accessibility contrast?
6. Why might you use a nested Navigator in a tab?

Submission and self-check

Submit two running navigation examples, themed screenshots and a concise primary/secondary navigation rationale.

CSC303 / NOVEMBER 2026 REVISION

LAB 07 EXTENSION | Midterm practical acceptance specification

The original 25-mark midterm allocation is unchanged. The observable checks below make the grading repeatable without tying students to exactly one sample application.

Criterion	Marks	Observable acceptance
Working features	12	Two screens (4); validated input/search (3); 15+ records (3); working dialog (2)
UI and accessibility	5	Readable portrait/landscape (2); feedback and empty state (2); labels (1)
Code structure	4	Appropriate widget decomposition (2); meaningful naming/state ownership (2)
Debugging and viva	4	Analyze passes (2); demonstrates a failure/fix and explains design (2)

Timed sequence

- Start from an approved starter project and a branch made during the assessment; do not spend the examination period installing Android Studio.
- Implement a small, independently functioning screen before adding secondary UI polish.
- Commit and record the final SHA; run `flutter analyze` and demonstrate on the phone.
- Show one negative input case, one empty or missing record case, and one navigation action with back behavior.

Fairness and contingency

The instructor should provide the same dataset, starter dependencies, phone policy and offline fallback to all candidates. A pre-existing SDK-download failure should be recorded separately from a student implementation defect. Students must still explain code they submit.

Viva probe examples

Which widget owns the filter state? What happens when the result list is empty? Why is a stable key useful when reordering list rows? How would you prevent a route from returning an invalid object?

LAB 08 Shared application state: Provider and Cubit/BLoC

Unit 3 • 3-hour guided laboratory • CLO-5

Learning goal. Manage state shared between widgets with reactive observables and compare Provider's ChangeNotifier with flutter_bloc's Cubit.

Core concepts. Ephemeral vs app state; ChangeNotifier, context.watch/read, Cubit, BlocBuilder, immutable state and rebuild boundaries.

Before you start. Flutter app compiling from Lab 07; install packages separately for each task.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab08
cd mad_lab08
flutter pub add provider flutter_bloc
```

Solved task 1 | Shopping cart using Provider

Problem and objective. Share one cart model between a catalog and a summary area without threading callbacks through every widget.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() => runApp(ChangeNotifierProvider(
  create: (_) => CartModel(),
  child: const MaterialApp(home: ShopPage()),
));

class CartModel extends ChangeNotifier {
  final Map<String, int> quantities = {};
  int get count => quantities.values.fold(0, (a, b) => a + b);
  void add(String item) {
    quantities[item] = (quantities[item] ?? 0) + 1;
    notifyListeners();
  }
  void clear() { quantities.clear(); notifyListeners(); }
}

class ShopPage extends StatelessWidget {
  const ShopPage({super.key});
  @override
  Widget build(BuildContext context) {
    const items = ['Notebook', 'Marker', 'Pen', 'Folder'];
    final count = context.watch<CartModel>().count;
    return Scaffold(
      appBar: AppBar(title: Text('Campus shop • Cart: $count')),
      body: ListView(children: [
        for (final item in items)
          ListTile(title: Text(item),
            trailing: FilledButton(
              onPressed: () => context.read<CartModel>().add(item),
              child: const Text('Add'))),
        const Divider(),
        Consumer<CartModel>(builder: (_, cart, __) => Column(
          children: [
            for (final entry in cart.quantities.entries)
              Text('${entry.key}: ${entry.value}'),
            TextButton(onPressed: cart.clear, child: const Text('Clear cart')),
          ],
        )),
      ]),
    );
  }
}
```

How the solution works. The ChangeNotifier model stores shared mutable data and calls notifyListeners after each update. context.watch subscribes to state, while context.read invokes an action without subscribing. Consumer restricts one part of the tree to cart-specific rebuilds.

Expected output. Adding several items changes the cart count and per-item quantities; Clear empties it.

Check your work. Place a breakpoint in build and observe why listeners, not direct field changes alone, cause a rebuild.

Solved task 2 | Cubit: a predictable attendance counter

Problem and objective. Use flutter_bloc's Cubit to emit immutable integer states and update the UI with BlocBuilder.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

class AttendanceCubit extends Cubit<int> {
  AttendanceCubit() : super(0);
  void markPresent() => emit(state + 1);
  void reset() => emit(0);
}

void main() => runApp(BlocProvider(
  create: (_) => AttendanceCubit(),
  child: const MaterialApp(home: AttendancePage()),
));

class AttendancePage extends StatelessWidget {
  const AttendancePage({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Attendance • Cubit')),
    body: Center(child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        BlocBuilder<AttendanceCubit, int>(builder: (_, count) =>
          Text('Present students: $count',
            style: const TextStyle(fontSize: 24))),
        const SizedBox(height: 16),
        FilledButton(
          onPressed: () => context.read<AttendanceCubit>().markPresent(),
          child: const Text('Mark present'),
        ),
        TextButton(
          onPressed: () => context.read<AttendanceCubit>().reset(),
          child: const Text('Reset'),
        ),
      ],
    )),
  );
}
```

How the solution works. Cubit is a simpler member of the BLoC family: UI invokes methods, the Cubit emits a new state, and BlocBuilder redraws the consuming subtree. A full Bloc normally maps explicit events to states. Neither Cubit nor Provider inherently makes in-memory data persistent.

Expected output. Present count increments with each press and Reset returns it to zero.

Check your work. Explain why emitting state + 1 is easier to trace than mutating a shared global variable.

Scenario-based practice — unsolved

Scenario 1. Attendance across screens. Synchronize present count between a dashboard, class list and analytics page; decide between Provider or Cubit and justify state ownership.

Scenario 2. Shopping cart scalability. Model add/remove quantity, a coupon and asynchronous stock checks. Describe a test plan and which state should be immutable.

Technical viva — justify your reasoning

1. What does notifyListeners actually notify?
2. Why should widgets use read for button callbacks but watch when displaying derived data?
3. What distinction separates Cubit from a full event-driven Bloc?
4. What is a rebuild boundary and why does it matter?
5. When would you avoid global shared state entirely?
6. How does state survive a process restart?

Submission and self-check

Submit both runnable examples, a state-flow diagram and a comparison of Provider vs Cubit (3–5 sentences).

LAB 09 On-device persistence: preferences and relational SQLite with Drift

Unit 4 • 3-hour guided laboratory • CLO-5

Learning goal. Persist small key-value data and build a relational table with reactive CRUD that survives app restarts.

Core concepts. SharedPreferencesAsync, SQL tables and primary keys, Drift typed schema, code generation, insert/select/delete, reactive streams and migrations.

Before you start. Use only non-sensitive preferences for small demo data. For real confidential data use appropriate secure-storage design and encryption.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab09
cd mad_lab09
flutter pub add shared_preferences drift drift_flutter
flutter pub add --dev drift_dev build_runner
```

Solved task 1 | Persist a reading goal with SharedPreferencesAsync

Problem and objective. Increment the number of completed reading sessions and retain the value after fully closing and reopening the app. This exercise uses the current asynchronous preferences API.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MaterialApp(home: ReadingGoalPage()));

class ReadingGoalPage extends StatefulWidget {
  const ReadingGoalPage({super.key});
  @override
  State<ReadingGoalPage> createState() => _ReadingGoalPageState();
}

class _ReadingGoalPageState extends State<ReadingGoalPage> {
  final prefs = SharedPreferencesAsync();
  int sessions = 0;
  bool loading = true;

  @override
  void initState() { super.initState(); load(); }

  Future<void> load() async {
    final value = await prefs.getInt('reading_sessions') ?? 0;
    if (!mounted) return;
    setState(() { sessions = value; loading = false; });
  }

  Future<void> addSession() async {
    final next = sessions + 1;
    await prefs.setInt('reading_sessions', next);
    if (mounted) setState(() => sessions = next);
  }

  Future<void> reset() async {
    await prefs.remove('reading_sessions');
    if (mounted) setState(() => sessions = 0);
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Reading tracker')),
    body: Center(child: loading
      ? const CircularProgressIndicator()
      : Column(mainAxisSize: MainAxisSize.min, children: [
          Text('$sessions sessions completed',
            style: Theme.of(context).textTheme.headlineSmall),
          FilledButton(onPressed: addSession,
            child: const Text('Complete session')),
          TextButton(onPressed: reset, child: const Text('Reset')),
        ])),
  );
```

```
}
```

How the solution works. SharedPreferencesAsync reads and writes a platform key-value store without a stale in-process cache. initState starts loading the saved value, and mounted checks protect against an asynchronous reply arriving after the widget is disposed.

Expected output. Increment the counter, force-close the app and reopen; the saved number returns. Reset removes the stored key.

Check your work. Confirm values survive a process restart but not necessarily an app uninstall or device data clearance.

Solved task 2 | Drift database: add, watch and delete notes

Problem and objective. Build a local relational SQLite-backed notes table, demonstrate reactive streams and generate type-safe database bindings. The generated file database.g.dart is produced by build_runner, not typed by students.

lib/database.dart

```
import 'package:drift/drift.dart';
import 'package:drift_flutter/drift_flutter.dart';

part 'database.g.dart';

class Notes extends Table {
  IntColumn get id => integer().autoIncrement()();
  TextColumn get content => text().withLength(min: 1, max: 200)();
}

@DriftDatabase(tables: [Notes])
class AppDatabase extends _$AppDatabase {
  AppDatabase() : super(driftDatabase(name: 'csc303_notes'));
  @override
  int get schemaVersion => 1;

  Stream<List<Note>> watchNotes() => select(notes).watch();
  Future<int> createNote(String content) =>
      into(notes).insert(NotesCompanion.insert(content: content));
  Future<int> deleteNote(int id) =>
      (delete(notes)..where((t) => t.id.equals(id))).go();
}
```

lib/main.dart

```
import 'package:flutter/material.dart';
import 'database.dart';

void main() => runApp(const MaterialApp(home: NotesPage()));

class NotesPage extends StatefulWidget {
  const NotesPage({super.key});
  @override
  State<NotesPage> createState() => _NotesPageState();
}

class _NotesPageState extends State<NotesPage> {
  final db = AppDatabase();
  final input = TextEditingController();

  @override
  void dispose() { input.dispose(); db.close(); super.dispose(); }

  Future<void> addNote() async {
    final text = input.text.trim();
    if (text.isEmpty || text.length > 200) return;
    await db.createNote(text);
    input.clear();
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Drift notes')),
    body: Column(children: [
      Padding(padding: const EdgeInsets.all(12), child: Row(children: [
        Expanded(child: TextField(controller: input,
          decoration: const InputDecoration(labelText: 'New note'))),
        IconButton(onPressed: addNote, icon: const Icon(Icons.add)),
      ])),
      Expanded(child: StreamBuilder<List<Note>>(
```

```
        stream: db.watchNotes(),
        builder: (context, snapshot) {
          if (snapshot.hasError) return Text('Error: ${snapshot.error}');
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
          final values = snapshot.data!;
          return ListView.builder(
            itemCount: values.length,
            itemBuilder: (_, i) => ListTile(
              title: Text(values[i].content),
              trailing: IconButton(
                icon: const Icon(Icons.delete_outline),
                onPressed: () => db.deleteNote(values[i].id)),
              ),
            );
        },
      )),
    ]),
  );
}
```

TERMINAL — generate before running

```
dart run build_runner build --delete-conflicting-outputs
flutter analyze
flutter run
```

How the solution works. The Notes class describes an SQL table with a primary key and constrained text. Drift generates _$AppDatabase and data class Note, enforcing types at compile time. The watchNotes query emits whenever underlying table data changes; StreamBuilder rebuilds the visible rows. schemaVersion must be incremented and migrations implemented for later schema changes.

Expected output. The app can add and delete note records; notes remain after force-closing and restarting the app.

Check your work. Confirm the generated database.g.dart exists. Task 2 covers create, watch/read and delete. Complete the update method in the following extension page to demonstrate all four CRUD operations.

FIELD NOTE • Drift generated code is required to build, and must be regenerated after a schema change. Generated Dart code may be committed to a team repository if your agreed policy requires reproducible builds; database files and credentials should not be committed.

Scenario-based practice — unsolved

Scenario 1. Offline expense database. Design tables for Expense and Category with a category foreign key and report monthly totals. Identify required schema constraints and migration risks.

Scenario 2. Preference migration. A preference key changes from themeDark (bool) to themeMode (string). Describe how to migrate existing user data safely and handle missing/invalid old values.

Technical viva — justify your reasoning

1. Why not store sensitive passwords in SharedPreferences?
2. What does a primary key guarantee?
3. How do stream-based queries differ from polling a database?
4. Why can changing Drift schema without migration destroy or hide existing data?
5. When is a key-value store preferable to SQL?
6. What is build_runner generating and why does the app need it?

Submission and self-check

Submit persistence test evidence (before/after restart), generated-code command, the database schema and a commit with a working notes UI.

CSC303 / NOVEMBER 2026 REVISION

LAB 09 EXTENSION | Complete the Drift CRUD requirement

Task 2 currently implements create, read/watch and delete. Treat its description as **CRD** until the update method below is added. The original stream and table do not need to be rewritten.

Step 1 | Add an update method to lib/database.dart

```
Future<int> updateNote(int id, String content) =>
    (update(notes)..where((t) => t.id.equals(id))).write(
      NotesCompanion(content: Value(content)),
    );
```

Place this method inside `AppDatabase`, alongside `createNote` and `deleteNote`. The existing `package:drift/drift.dart` import supplies `Value`.

Step 2 | Add an edit dialog inside _NotesPageState

```
Future<void> editNote(Note note) async {
  var draft = note.content;
  final result = await showDialog<String>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: const Text('Edit note'),
      content: TextFormField(
        initialValue: note.content,
        onChanged: (value) => draft = value,
      ),
      actions: [
        TextButton(onPressed: () => Navigator.pop(dialogContext),
            child: const Text('Cancel')),
        FilledButton(onPressed: () => Navigator.pop(dialogContext, draft),
            child: const Text('Save')),
      ],
    ),
  );
  if (!mounted || result == null) return;
  final changed = result.trim();
  if (changed.isEmpty || changed.length > 200) return;
  await db.updateNote(note.id, changed);
}
```

In the existing ListTile builder, add `onTap: () => editNote(values[i]),`. This extends, rather than replaces, the preceding Lab 09 source code.

Checks that demonstrate all four operations

- Create two notes; verify both appear in the stream.
- Tap a note, edit and save; verify its visible text changes without `setState` for the list.
- Cancel editing and verify no update; test empty/201-character input.
- Delete one note, force-close the app, reopen and confirm persisted state.

SQLite/Drift is the required local relational storage path in this task. Preferences remain appropriate only for small, non-confidential values.

LAB 10 HTTP REST APIs, JSON and offline fallback

Unit 4 • 3-hour guided laboratory • CLO-5

Learning goal. Retrieve remote JSON with robust error/loading states and preserve a last-successful response for offline reading.

Core concepts. http Client, HTTPS and Android INTERNET permission, JSON mapping, FutureBuilder, timeouts, HTTP status codes, shared_preferences cache.

Before you start. A phone with internet access; network API examples use public JSONPlaceholder https://jsonplaceholder.typicode.com and may fail under restrictive networks.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab10
cd mad_lab10
flutter pub add http shared_preferences
```

Android network permission

For both tasks, add the following line DIRECTLY inside the <manifest ...> element and outside the <application> element of android/app/src/main/AndroidManifest.xml. Preserve other manifest content; do not replace the generated file wholesale.

android/app/src/main/AndroidManifest.xml — insert inside <manifest>

```
<uses-permission android:name="android.permission.INTERNET" />
```

Solved task 1 | Fetch and parse live posts using a typed model

Problem and objective. Call a GET endpoint, check its status, decode a JSON array into Post objects, and expose loading, success and failure states.

lib/main.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

class Post {
  final int id;
  final String title;
  const Post(this.id, this.title);
  factory Post.fromJson(Map<String, dynamic> json) =>
      Post(json['id'] as int, json['title'] as String);
}

Future<List<Post>> fetchPosts() async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'))
      .timeout(const Duration(seconds: 12));
  if (response.statusCode != 200) {
    throw Exception('Server returned ${response.statusCode}');
  }
  final values = jsonDecode(response.body) as List<dynamic>;
  return values.map((e) => Post.fromJson(e as Map<String, dynamic>))
      .toList();
}

void main() => runApp(const MaterialApp(home: PostsPage()));

class PostsPage extends StatefulWidget {
  const PostsPage({super.key});
  @override
  State<PostsPage> createState() => _PostsPageState();
}

class _PostsPageState extends State<PostsPage> {
  late Future<List<Post>> request;
  @override
  void initState() { super.initState(); request = fetchPosts(); }

  @override
  Widget build(BuildContext context) => Scaffold(
```

```
    appBar: AppBar(title: const Text('REST posts')),
    floatingActionButton: FloatingActionButton(
      onPressed: () => setState(() => request = fetchPosts()),
      child: const Icon(Icons.refresh)),
    body: FutureBuilder<List<Post>>(
      future: request,
      builder: (context, snapshot) {
        if (snapshot.connectionState != ConnectionState.done) {
          return const Center(child: CircularProgressIndicator());
        }
        if (snapshot.hasError) {
          return Center(child: Text('Request failed: ${snapshot.error}'));
        }
        final posts = snapshot.data ?? [];
        return ListView.builder(itemCount: posts.length,
          itemBuilder: (_, i) => ListTile(
            leading: CircleAvatar(child: Text('${posts[i].id}')),
            title: Text(posts[i].title),
          ));
      },
    ),
  );
}
```

How the solution works. HTTP returns bytes/text and a status code; jsonDecode transforms text into Dart structures, and the factory enforces a consistent typed model. A stored Future in State prevents repeated HTTP calls every time build executes. timeout avoids indefinite waiting and FutureBuilder describes the asynchronous lifecycle.

Expected output. A scrolling list of sample posts appears online; airplane mode plus Refresh produces a friendly error, not an uncaught crash.

Check your work. Record behavior with the network available and unavailable; identify the status code handling branch.

Solved task 2 | Offline-first bulletin cache

Problem and objective. Save the last successful bulletin response in preferences. When the API fails, show the cached copy with an explicit offline label.

lib/main.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MaterialApp(home: OfflinePage()));

class OfflinePage extends StatefulWidget {
  const OfflinePage({super.key});
  @override
  State<OfflinePage> createState() => _OfflinePageState();
}

class _OfflinePageState extends State<OfflinePage> {
  final prefs = SharedPreferencesAsync();
  List<String> titles = [];
  String status = 'Loading...';
  bool busy = false;

  @override
  void initState() { super.initState(); refresh(); }

  Future<void> refresh() async {
    if (busy) return;
    setState(() => busy = true);
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts'))
          .timeout(const Duration(seconds: 10));
      if (response.statusCode != 200) throw Exception('HTTP error');
      final decoded = jsonDecode(response.body) as List<dynamic>;
      final latest = decoded.take(15).map((e) =>
        (e as Map<String, dynamic>)['title'] as String).toList();
      await prefs.setStringList('cached_bulletins', latest);
      if (mounted) setState(() {
        titles = latest; status = 'Online • Updated from API';
      });
    } catch (_) {
```

```
      final cached = await prefs.getStringList('cached_bulletins') ?? [];
      if (mounted) setState(() {
        titles = cached;
        status = cached.isEmpty ? 'Offline • No cached data yet'
            : 'Offline • Showing saved copy';
      });
    } finally {
      if (mounted) setState(() => busy = false);
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Offline bulletins')),
    body: Column(children: [
      ListTile(title: Text(status),
        trailing: IconButton(onPressed: busy ? null : refresh,
          icon: const Icon(Icons.refresh))),
      if (busy) const LinearProgressIndicator(),
      Expanded(child: ListView.builder(
        itemCount: titles.length,
        itemBuilder: (_, i) => ListTile(title: Text(titles[i])),
      )),
    ]),
  );
}
```

How the solution works. The network is authoritative when reachable. After a validated successful response, the app caches 15 titles. The exception path reads the last saved values. Users see a data-freshness indicator so old information is not mistaken for fresh data. Persistent cache is a fallback, not a full synchronization engine.

Expected output. Open online once; then enable airplane mode, press Refresh and see saved titles labeled Offline. On a clean offline install, see No cached data yet.

Check your work. Test online-first, offline-after-cache and first-run-without-internet separately.

API correctness and security review

Treat API responses as untrusted: check status and type, handle timeout and decoding errors, use HTTPS, do not embed secret API tokens in the Flutter client, and distinguish connection failure from unauthorized response. If the remote contract changes, change the model parser and add tests before shipping.

Scenario-based practice — unsolved

Scenario 1. Bus timetable cache. The timetable must work without internet and warn users if data is older than one day. Specify a timestamp schema, refresh policy and conflict behavior.

Scenario 2. Account-specific API. Consume a bearer-token endpoint without placing long-lived secrets in app source. Identify storage, token refresh, HTTP 401 handling and logout behavior.

Technical viva — justify your reasoning

1. Why is an HTTP 200 response not a guarantee of valid JSON shape?
2. Why should a Future not be created directly on every build?
3. What distinguishes offline-first from online-only with a spinner?
4. Why can caching stale medical or emergency data be dangerous?
5. What is the security difference between an API key baked into an app and a server-held key?
6. Where should Android INTERNET permission be declared?

Submission and self-check

Demonstrate both online fetch and cached offline fallback, with Android permission and a short error-state test log.

CSC303 / NOVEMBER 2026 REVISION

LAB 10 EXTENSION | Correct cache semantics and error tests

The solved network examples implement **network-first with fallback**. Offline-first means the UI is driven by the local store and network synchronization occurs separately. Do not label a timeout-then-cache flow as a complete offline-first design.

Safe ordering for a last-known-good cache

```
Future<List<Post>> fetchValidatedPosts(http.Client client) async {
  final response = await client.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'),
  ).timeout(const Duration(seconds: 10));
  if (response.statusCode != 200) {
    throw Exception('HTTP ${response.statusCode}');
  }
  final decoded = jsonDecode(response.body) as List<dynamic>;
  final posts = decoded.take(25).map((row) =>
      Post.fromJson(row as Map<String, dynamic>)).toList();
  // Now, and only now, save response.body as the new cache.
  return posts;
}
```

This is a pattern: adapt the model name `Post` to the actual model from the task. Validate the required fields and types, not just the HTTP code. Keep a usable cache when parsing fails, and preserve a status indicator explaining whether data is fresh or cached.

Minimum network and persistence test matrix

Case	Expected result
Valid HTTP 200 with expected JSON	Display typed records and update cache
HTTP 500 / timeout / disconnected	Display previous valid cache, visibly marked stale
HTTP 200 with malformed JSON	Do NOT replace previous cache; show fallback/error
No network and no cache	Render an informative empty/error state; retry is available
App restart while offline	Persisted cache remains accessible
Unexpected fields / missing required field	Handle as an invalid record/response without an uncaught crash

Implementation boundary

For the final project, put the HTTP client behind an injectable repository and persist typed records in Drift/SQLite or Firestore as required by the project brief. SharedPreferencesAsync is acceptable for a demonstration cache but does not replace the required relational/cloud storage layer in the final project.

LAB 11 Firebase Authentication: signup, login and session lifecycle

Unit 4 • 3-hour guided laboratory • CLO-5

Learning goal. Connect an Android Flutter app to a real Firebase project, register users, sign in, sign out and send password-reset email.

Core concepts. Firebase project registration, FlutterFire CLI, generated firebase_options.dart, Auth exceptions, authStateChanges stream, secure identity concepts.

Before you start. Working internet, personal Firebase project and authorized Android device. Cloud authentication requires service-side configuration; source code cannot replace that configuration.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android --org edu.cui.sahiwal mad_lab11
cd mad_lab11
flutter pub add firebase_core firebase_auth
# Install Firebase CLI following the official OS-specific instructions.
# Then run: firebase login
dart pub global activate flutterfire_cli
flutterfire configure
```

Firebase console and CLI prerequisites

1. Create/select a project at https://console.firebase.google.com. In Authentication > Sign-in method enable Email/Password. Avoid any real student passwords in demo accounts; use unique test credentials.
2. Install the Firebase CLI using the method for your operating system (https://firebase.google.com/docs/cli); sign in with firebase login. Install the FlutterFire CLI and ensure Dart's pub global binaries folder is in PATH if flutterfire is not found.
3. Execute flutterfire configure in the Flutter app root and choose Android and the correct Firebase project. This generates lib/firebase_options.dart and updates the relevant Android integration. Run it again after adding Firebase services/plugins as required.
4. A generated firebase_options.dart is project-specific, so it cannot be supplied as a universal copy-paste source file. Both solved app files below are complete once the documented generator has created that configuration.

Solved task 1 | Email/password registration and login with auth gate

Problem and objective. Create one app that supports both signup and login and switches automatically to an authenticated home screen. Do not log or save a plaintext password.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform);
  runApp(const MaterialApp(home: AuthGate()));
}

class AuthGate extends StatelessWidget {
  const AuthGate({super.key});
  @override
  Widget build(BuildContext context) => StreamBuilder<User?>(
    stream: FirebaseAuth.instance.authStateChanges(),
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Scaffold(body: Center(child: CircularProgressIndicator()));
      }
      return snapshot.data == null ? const LoginPage() : const MemberPage();
    },
  );
```

```
}

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});
  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final email = TextEditingController();
  final password = TextEditingController();
  String message = '';
  bool busy = false;
  @override
  void dispose() { email.dispose(); password.dispose(); super.dispose(); }

  Future<void> submit({required bool register}) async {
    if (email.text.trim().isEmpty || password.text.length < 6) {
      setState(() => message = 'Email and 6+ character password required');
      return;
    }
    setState(() => busy = true);
    try {
      if (register) {
        await FirebaseAuth.instance.createUserWithEmailAndPassword(
          email: email.text.trim(), password: password.text);
      } else {
        await FirebaseAuth.instance.signInWithEmailAndPassword(
          email: email.text.trim(), password: password.text);
      }
    } on FirebaseAuthException catch (e) {
      if (mounted) setState(() => message = 'Authentication: ${e.code}');
    } finally {
      if (mounted) setState(() => busy = false);
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Firebase login / signup')),
    body: ListView(padding: const EdgeInsets.all(20), children: [
      TextField(controller: email,
        keyboardType: TextInputType.emailAddress,
        decoration: const InputDecoration(labelText: 'Email')),
      TextField(controller: password, obscureText: true,
        decoration: const InputDecoration(labelText: 'Password')),
      const SizedBox(height: 16),
      FilledButton(onPressed: busy ? null : () => submit(register: false),
        child: const Text('Sign in')),
      OutlinedButton(onPressed: busy ? null : () => submit(register: true),
        child: const Text('Create account')),
      Text(message),
    ]),
  );
}

class MemberPage extends StatelessWidget {
  const MemberPage({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Member area')),
    body: Center(child: Column(mainAxisSize: MainAxisSize.min,
      children: [
        Text('Signed in: ${FirebaseAuth.instance.currentUser?.email}'),
        FilledButton(onPressed: () => FirebaseAuth.instance.signOut(),
          child: const Text('Sign out')),
      ],
    )),
  );
}
```

TERMINAL — after FlutterFire configuration

```
flutter pub get
flutter analyze
flutter run
```

How the solution works. Firebase.initializeApp sets the configured backend, and authStateChanges is a stream that emits login/logout state. The UI is gated by that stream rather than a manually stored isLoggedIn boolean. FirebaseAuthException provides codes for duplicate email, invalid credentials, disabled accounts or network failure.

Expected output. Register a test account; the member screen shows the signed-in email. Sign out and sign back in.

Check your work. Try duplicate registration and wrong password; report sanitized error codes rather than exposing secrets.

Solved task 2 | Member profile, email verification and password reset

Problem and objective. Show an authenticated profile and provide logout, verification and reset actions. For a reset demonstration, enter an email address, call the backend and display a non-revealing response.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform);
  runApp(const MaterialApp(home: AccountPage()));
}

class AccountPage extends StatefulWidget {
  const AccountPage({super.key});
  @override
  State<AccountPage> createState() => _AccountPageState();
}

class _AccountPageState extends State<AccountPage> {
  final email = TextEditingController();
  String feedback = 'Sign in using Task 1, or provide an email to reset.';
  @override
  void dispose() { email.dispose(); super.dispose(); }

  Future<void> reset() async {
    final value = email.text.trim();
    if (!value.contains('@')) {
      setState(() => feedback = 'Enter a valid email format'); return;
    }
    try {
      await FirebaseAuth.instance.sendPasswordResetEmail(email: value);
      if (mounted) setState(() => feedback =
        'If an account is eligible, a reset email will be sent.');
    } on FirebaseAuthException {
      if (mounted) setState(() => feedback =
        'Reset unavailable; check email settings or try later.');
    }
  }

  Future<void> verify() async {
    final user = FirebaseAuth.instance.currentUser;
    if (user == null) return;
    try {
      await user.sendEmailVerification();
      if (mounted) setState(() => feedback = 'Check your inbox.');
    } on FirebaseAuthException catch (e) {
      if (mounted) setState(() => feedback = 'Verification: ${e.code}');
    }
  }

  @override
```

```
  Widget build(BuildContext context) {
    final user = FirebaseAuth.instance.currentUser;
    return Scaffold(
      appBar: AppBar(title: const Text('Account tools')),
      body: ListView(padding: const EdgeInsets.all(20), children: [
        Text('Current user: ${user?.email ?? 'Not signed in'}'),
        Text('Verified: ${user?.emailVerified ?? false}'),
        TextField(controller: email,
          decoration: const InputDecoration(labelText: 'Reset email')),
        FilledButton(onPressed: reset,
          child: const Text('Send password reset')),
        OutlinedButton(onPressed: user == null ? null : verify,
          child: const Text('Send verification email')),
        OutlinedButton(onPressed: user == null ? null
          : () async { await FirebaseAuth.instance.signOut();
              if (mounted) setState(() {}); },
          child: const Text('Sign out')),
        Text(feedback),
      ]),
    );
  }
}
```

How the solution works. Password resets are performed by Firebase, not by retrieving a stored password. A neutral completion message helps avoid leaking account existence to observers. Verification is only available with a current signed-in user. This exercise demonstrates direct auth API usage; Task 1's auth gate is the preferable way to coordinate navigation after sign-out.

Expected output. A signed-in user can request email verification and sign out. Reset requests can be initiated for a known test email.

Check your work. Check Firebase email templates and that test messages arrive; check the app also handles a network error gracefully.

Security and policy notes

Firebase client config contains application identifiers and is not a server-side secret, but Authentication, Firestore and Storage still need correctly scoped security rules. Email/password credentials belong to the identity provider; never place the password in shared_preferences, source control, application logs or a screenshot. Add account deletion, rate limiting, verification and abuse-monitoring procedures in production.

Scenario-based practice — unsolved

Scenario 1. Clinic appointment identity. Design sign-up, email verification, forgot-password and account-deletion flows without revealing patient data to logged-out users. Identify the backend rule boundaries.

Scenario 2. Shared family phone. A second student opens the app after another student used it. Explain persistent login, sign-out, reauthentication for critical actions and data isolation.

Technical viva — justify your reasoning

1. Why is authStateChanges safer than a local isLoggedIn flag?
2. What is the difference between client-side input validation and Firebase backend authorization?
3. Why should account-existence errors be carefully surfaced in reset flows?
4. Can firebase_options.dart enforce private database access? Why not?
5. Why should authenticated UI be rebuilt after sign-out?
6. What additional security controls would be required for high-value records?

Submission and self-check

Submit registration, login, logout and reset evidence without exposing passwords, plus Firebase test-account and failure-state screenshots.

CSC303 / NOVEMBER 2026 REVISION

LAB 11 EXTENSION | A complete authentication journey

Task 1 demonstrates an auth gate; Task 2 demonstrates account actions in a separate replacement application. For this extension, make Task 2 reachable from the signed-in area of Task 1 rather than launching AccountPage as a stand-alone root.

Integrated navigation and session behavior

• Start the app signed out; show registration/login. Register a designated test account. Use `authStateChanges()` to switch from login to authenticated content.

• Inside authenticated content, navigate to `AccountPage` and expose reset-email and verification actions. After sign-out, the auth gate returns to the signed-out experience.

• Repeat after force-close/relaunch to confirm whether Firebase restores the session; never use a locally stored boolean as the source of truth.

Refresh verification status after returning from email

```
Future<void> refreshVerification() async {
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) return;
  try {
    await user.reload();
    if (!mounted) return;
    setState(() {}); // Re-read currentUser in build().
  } on FirebaseAuthException catch (e) {
    if (mounted) setState(() => feedback = e.code);
  }
}
```

Add a `Refresh verification status` button bound to this method. The original display of `emailVerified` can be stale until the current user is refreshed.

Security assertions

• Unauthenticated screens must not display a previous user's private records; navigation hiding is not backend authorization.

• Do not log passwords, private user data or authentication tokens; use reset messages that do not enumerate whether accounts exist.

• Document whether account deletion, reauthentication and abuse controls are excluded from the classroom demo or implemented.

Four acceptance demonstrations

Signed-out startup; successful login and restart; verification refresh; sign-out with private data removed from the UI. Include one invalid-password case and one network failure.

LAB 12 Cloud Firestore, Firebase Storage and push messaging

Unit 4 • 3-hour guided laboratory • CLO-5

Learning goal. Read/write authenticated cloud records in real time, upload an image to controlled cloud storage, and receive Firebase Cloud Messaging foreground events.

Core concepts. Firestore snapshots, document IDs and security rules; Firebase Storage object paths; image_picker; FCM token and foreground listener.

Before you start. Use the SAME Firebase project configured for Lab 11, but a separate Flutter demo project. Enable Anonymous sign-in for the exercises. FIREBASE CLOUD STORAGE currently requires a Blaze (pay-as-you-go) project; do not activate billing without instructor approval.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android --org edu.cui.sahiwal mad_lab12
cd mad_lab12
flutter pub add firebase_core firebase_auth cloud_firestore firebase_storage firebase_messaging
flutter pub add image_picker
dart pub global activate flutterfire_cli
flutterfire configure
```

Required Firebase console configuration and safe rules

Enable Authentication > Sign-in method > Anonymous for the demo. Create a Firestore database. Publish the following example rules in Firestore Rules so each anonymous signed-in user reads/writes only their own notes. For production, validate fields, sizes and app authorization more tightly. New anonymous identities may not survive app-data clearing.

FIRESTORE SECURITY RULES — Firebase console, not lib/main.dart

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/notes/{noteId} {
      allow read, write: if request.auth != null
        && request.auth.uid == userId;
    }
  }
}
```

Solved task 1 | User-scoped Firestore notes with real-time updates

Problem and objective. Write notes to users/<uid>/notes and observe live database updates from StreamBuilder. Anonymous sign-in is enabled explicitly for this classroom demo.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform);
  await FirebaseAuth.instance.signInAnonymously();
  runApp(const MaterialApp(home: CloudNotesPage()));
}

class CloudNotesPage extends StatefulWidget {
  const CloudNotesPage({super.key});
  @override
  State<CloudNotesPage> createState() => _CloudNotesPageState();
}

class _CloudNotesPageState extends State<CloudNotesPage> {
  final input = TextEditingController();
  @override
  void dispose() { input.dispose(); super.dispose(); }
```

```
  CollectionReference<Map<String, dynamic>> get notes {
    final uid = FirebaseAuth.instance.currentUser!.uid;
    return FirebaseFirestore.instance.collection('users')
      .doc(uid).collection('notes');
  }

  Future<void> add() async {
    final value = input.text.trim();
    if (value.isEmpty) return;
    try {
      await notes.add({'body': value,
        'createdAt': FieldValue.serverTimestamp()});
      input.clear();
    } catch (e) {
      if (mounted) ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Save failed: $e')));
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Firestore notes')),
    body: Column(children: [
      Padding(padding: const EdgeInsets.all(12), child: Row(children: [
        Expanded(child: TextField(controller: input,
          decoration: const InputDecoration(labelText: 'Cloud note'))),
        IconButton(onPressed: add, icon: const Icon(Icons.cloud_upload)),
      ])),
      Expanded(child: StreamBuilder<QuerySnapshot<Map<String, dynamic>>>(
        stream: notes.snapshots(),
        builder: (context, snapshot) {
          if (snapshot.hasError) return Text('Firestore error: ${snapshot.error}');
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
          final docs = snapshot.data!.docs;
          return ListView.builder(itemCount: docs.length,
            itemBuilder: (context, i) {
              final row = docs[i];
              return ListTile(
                title: Text(row.data()['body'] as String? ?? ''),
                trailing: IconButton(icon: const Icon(Icons.delete),
                  onPressed: () => row.reference.delete()),
              );
            });
        },
      )),
    ]),
  );
}
```

How the solution works. The authenticated Firebase UID scopes the collection path. Firestore snapshots stream database changes to the app without manual polling. Each document has an auto-generated unique ID, and row.reference.delete() removes the selected document. Security is enforced by the Firestore Rules on the backend, not by hiding UI elements.

Expected output. Notes added from the phone appear in Firestore console and disappear live when deleted.

Check your work. Verify a second independently signed-in test user cannot read the first user's notes under the published rules.

Cloud Storage security rules and billing gate

Cloud Storage currently requires the Firebase Blaze (pay-as-you-go) plan. Before attempting Task 2, obtain instructor approval, enable billing/budget alerts, create the default bucket and then re-run flutterfire configure so lib/firebase_options.dart reflects the bucket. Publish the scoped rules below in Cloud Storage Rules. Never enable broad public write permissions. For production systems, also validate file size and content type.

CLOUD STORAGE SECURITY RULES — Firebase console

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /uploads/{uid}/{fileName} {
      allow read, write: if request.auth != null
        && request.auth.uid == uid;
    }
  }
```

```
}
```

Solved task 2 | Upload an image and inspect an FCM token/message

Problem and objective. Use image_picker to choose one gallery image, upload it under an authenticated user-scoped path and show its URL-based preview. Also request FCM permission, display the device token and listen for foreground messages. Storage upload requires the billing/rules gate above.

lib/main.dart

```
import 'dart:async';
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_storage/firebase_storage.dart';
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:image_picker/image_picker.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform);
  await FirebaseAuth.instance.signInAnonymously();
  runApp(const MaterialApp(home: MediaPage()));
}

class MediaPage extends StatefulWidget {
  const MediaPage({super.key});
  @override
  State<MediaPage> createState() => _MediaPageState();
}

class _MediaPageState extends State<MediaPage> {
  String status = 'No image selected';
  String? imageUrl;
  String token = 'Loading FCM token...';
  String message = 'No foreground push received';
  StreamSubscription<RemoteMessage>? subscription;

  @override
  void initState() { super.initState(); initMessaging(); }

  Future<void> initMessaging() async {
    try {
      final messaging = FirebaseMessaging.instance;
      await messaging.requestPermission();
      final value = await messaging.getToken();
      subscription = FirebaseMessaging.onMessage.listen((event) {
        if (mounted) setState(() => message =
          event.notification?.title ?? 'Data message received');
      });
      if (mounted) setState(() => token = value ?? 'No token yet');
    } catch (e) {
      if (mounted) setState(() => token = 'FCM unavailable: $e');
    }
  }

  Future<void> upload() async {
    final photo = await ImagePicker().pickImage(source: ImageSource.gallery);
    if (photo == null) return;
    setState(() => status = 'Uploading...');
    try {
      final uid = FirebaseAuth.instance.currentUser!.uid;
      final ref = FirebaseStorage.instance.ref()
        .child('uploads/$uid/${DateTime.now().millisecondsSinceEpoch}.jpg');
      await ref.putFile(File(photo.path));
      final url = await ref.getDownloadURL();
      if (mounted) setState(() {
        imageUrl = url; status = 'Upload complete';
      });
    } catch (e) {
      if (mounted) setState(() => status = 'Upload error: $e');
    }
  }

  @override
  void dispose() { subscription?.cancel(); super.dispose(); }
```

```
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Storage + FCM demo')),
    body: ListView(padding: const EdgeInsets.all(16), children: [
      FilledButton(onPressed: upload, child: const Text('Choose & upload image')),
      Text(status),
      if (imageUrl != null) Image.network(imageUrl!, height: 170),
      const Divider(),
      const Text('FCM device token (do not publish publicly):'),
      SelectableText(token, maxLines: 4),
      Text('Foreground notification: $message'),
    ]),
  );
}
```

How the solution works. image_picker supplies an XFile; putFile uploads a local image to a Firebase Storage object under uploads/<uid>/. This demonstration always writes a .jpg object name, so use a JPEG input or extend the example to preserve the real filename and MIME type before production use. A Firebase download URL can act as a shareable access token: do not paste it into public submissions. For FCM, getToken identifies this app instance and onMessage listens for foreground messages; treat tokens as sensitive test identifiers, never publish them.

Expected output. With Storage provisioned, an uploaded image appears and previews. On an FCM-capable phone, a token appears and a test foreground notification updates the label.

Check your work. Send a test from Firebase Console Cloud Messaging to a controlled test token; never store server credentials inside the Flutter application.

FCM delivery depends on Google Play services, Android notification permissions, app foreground/background behavior and the project's messaging configuration. A Flutter client alone cannot safely send administrative pushes: use Firebase Console for tests or a trusted backend using FCM HTTP v1 with authorized server credentials. Firestore does not itself send FCM messages.

Scenario-based practice — unsolved

Scenario 1. Group photo noticeboard. Users may only upload their own class images; staff can moderate and delete violating uploads. Define rules, object ownership, metadata validation and retention.

Scenario 2. Announcement notifications. Students subscribe to department announcements but must not see another department's private notices. Specify topic strategy, authenticated backend authorization, consent and token rotation.

Technical viva — justify your reasoning

1. Why are Firestore Rules more important than client-side hiding?
2. What is the difference between a document ID and a server timestamp?
3. Why does a streamed query usually not need polling?
4. What security risk follows globally readable Cloud Storage rules?
5. Why must FCM send credentials live only on a trusted backend?
6. What can prevent a device from obtaining an FCM token even when the Flutter code is correct?

Submission and self-check

Submit Firestore CRUD/stream evidence and rules; for Storage, show upload only if billing was approved. For FCM, redact token screenshots. Explain all external configuration steps.

CSC303 / NOVEMBER 2026 REVISION

LAB 12 EXTENSION | Safe cloud setup, permissions and fallback

Corrected package commands (supersede any split installation command in Lab 12):

```
flutter pub add firebase_core firebase_auth cloud_firestore
flutter pub add firebase_storage firebase_messaging image_picker
dart pub global activate flutterfire_cli
flutterfire configure
```

Run the commands from the `mad_lab12` app root, after creating it. Ensure the Dart global executable directory is on PATH before running `flutterfire`. The generated Firebase options are project/platform specific.

Instructor-only billing gate

Cloud Storage for Firebase requires Blaze billing. The instructor must approve, provision and monitor this service. A cost budget alert is a warning mechanism, not a guaranteed hard spending cap. Do not require students to enter payment methods for assessed work. Official reference: https://firebase.google.com/docs/storage/flutter/start.

Replace the example Storage rules with bounded writes

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /uploads/{uid}/{fileName} {
      allow read, delete: if request.auth != null
        && request.auth.uid == uid;
      allow create, update: if request.auth != null
        && request.auth.uid == uid
        && request.resource.size < 5 * 1024 * 1024
        && request.resource.contentType
             .matches('image/(jpeg|png|webp)');
    }
  }
}
```

Use a filename/extension matching the selected actual image type and provide consistent upload content-type metadata. The original example labels every upload .jpg even when a gallery picker may return a different format. Storage download URLs may be bearer links; do not publish them.

No-billing alternative and FCM testing

• If billing is not authorized: complete Task 1 Firestore scoped notes, then use an on-device image picker preview without upload. Record cloud upload as BLOCKED by policy, not failed.

• For FCM on Android 13+, request notification permission as needed, test foreground delivery from Firebase Console and redact registration tokens. A client token alone does not prove push delivery.

• Test a second anonymous user and verify that Firestore/Storage paths are isolated by UID. Rules, not widgets, enforce authorization.

LAB 13 MVVM architecture, repositories, dependency injection and testing

Unit 5 • 3-hour guided laboratory • CLO-5

Learning goal. Move business logic away from widgets, inject repository dependencies and verify behavior using automated Dart and Flutter widget tests.

Core concepts. Model/View/ViewModel separation, repository interfaces, constructor injection, ChangeNotifier, unit vs widget tests, deterministic fake data.

Before you start. Familiarity with ChangeNotifier and collections. Create separate projects for Task 1 and Task 2 or replace all named files.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab13
cd mad_lab13
```

Architecture overview

View = widget tree and user interactions; ViewModel = UI-facing state and actions; Repository = data access contract; Data source = network, local database or test fake. Dependency injection supplies a specific repository instance to the ViewModel. A ViewModel should not import MaterialApp or navigate using BuildContext unless the application design explicitly assigns it that responsibility.

SUGGESTED DIRECTORY TREE — Task 1

```
lib/
  main.dart                # View + application bootstrap
  data/course_repository.dart   # Contract and implementation
  viewmodel/course_viewmodel.dart  # State and business logic
```

Solved task 1 | Testable course browser using simple MVVM

Problem and objective. Separate course loading from widget rendering. Inject a repository instance through the ViewModel constructor rather than allowing the page to create network/database details.

lib/data/course_repository.dart

```
abstract class CourseRepository {
  Future<List<String>> loadCourses();
}

class DemoCourseRepository implements CourseRepository {
  @override
  Future<List<String>> loadCourses() async {
    await Future<void>.delayed(const Duration(milliseconds: 300));
    return ['Mobile App Development', 'Database Systems',
      'Software Testing', 'Operating Systems'];
  }
}
```

lib/viewmodel/course_viewmodel.dart

```
import 'package:flutter/foundation.dart';
import '../data/course_repository.dart';

class CourseViewModel extends ChangeNotifier {
  final CourseRepository repository;
  CourseViewModel(this.repository);
  List<String> courses = [];
  String? error;
  bool loading = false;

  Future<void> load() async {
    loading = true;
    error = null;
    notifyListeners();
```

```
    try {
      courses = await repository.loadCourses();
    } catch (e) {
      error = 'Unable to load courses: $e';
    } finally {
      loading = false;
      notifyListeners();
    }
  }
}
```

lib/main.dart

```
import 'package:flutter/material.dart';
import 'data/course_repository.dart';
import 'viewmodel/course_viewmodel.dart';

void main() => runApp(const MaterialApp(home: CourseScreen()));

class CourseScreen extends StatefulWidget {
  const CourseScreen({super.key});
  @override
  State<CourseScreen> createState() => _CourseScreenState();
}

class _CourseScreenState extends State<CourseScreen> {
  late final CourseViewModel model;

  @override
  void initState() {
    super.initState();
    model = CourseViewModel(DemoCourseRepository())..load();
  }

  @override
  void dispose() { model.dispose(); super.dispose(); }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('MVVM course list')),
    body: AnimatedBuilder(
      animation: model,
      builder: (context, _) {
        if (model.loading) {
          return const Center(child: CircularProgressIndicator());
        }
        if (model.error != null) return Center(child: Text(model.error!));
        return ListView.builder(
          itemCount: model.courses.length,
          itemBuilder: (_, i) => ListTile(title: Text(model.courses[i])),
        );
      },
    ),
  );
}
```

How the solution works. The repository contract isolates data operations; the ViewModel manages loading, data and error state; the View uses an AnimatedBuilder to listen for change notifications. Constructor injection allows replacement of DemoCourseRepository with an API repository or deterministic fake without rewriting the screen.

Expected output. A brief progress spinner transitions into four course rows without any networking.

Check your work. Temporarily throw Exception from DemoCourseRepository and verify the ViewModel reports a user-readable error state.

Solved task 2 | Unit tests + widget test with runnable app

Problem and objective. Practice red/green testing on pure score logic and widget behavior without connecting to external services. The Flutter template includes flutter_test as a dev dependency.

lib/course_math.dart

```
int totalScore({required int assignment,
    required int midterm, required int finalExam}) {
  if (assignment < 0 || assignment > 25 ||
      midterm < 0 || midterm > 25 ||
      finalExam < 0 || finalExam > 50) {
    throw ArgumentError('Marks out of range');
  }
  return assignment + midterm + finalExam;
}
```

lib/main.dart

```
import 'package:flutter/material.dart';
import 'course_math.dart';

void main() => runApp(const MaterialApp(home: ScorePage()));

class ScorePage extends StatefulWidget {
  const ScorePage({super.key});
  @override
  State<ScorePage> createState() => _ScorePageState();
}

class _ScorePageState extends State<ScorePage> {
  int score = 0;
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Score tests')),
    body: Center(child: Column(mainAxisSize: MainAxisSize.min,
      children: [
        Text('Score: $score', key: const Key('score_text')),
        FilledButton(
          onPressed: () => setState(() => score = totalScore(
            assignment: 20, midterm: 20, finalExam: 40)),
          child: const Text('Calculate'),
        ),
      ],
    )),
  );
}
```

test/course_math_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:mad_lab13/course_math.dart';

void main() {
  test('all maximum marks sum to 100', () {
    expect(totalScore(assignment: 25, midterm: 25,
        finalExam: 50), 100);
  });

  test('out-of-range marks are rejected', () {
    expect(() => totalScore(assignment: 26, midterm: 20,
      finalExam: 40), throwsArgumentError);
  });
}
```

test/widget_test.dart — REPLACE generated starter test

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mad_lab13/main.dart';

void main() {
  testWidgets('calculate button displays 80', (tester) async {
    await tester.pumpWidget(const MaterialApp(home: ScorePage()));
    expect(find.text('Score: 0'), findsOneWidget);
    await tester.tap(find.text('Calculate'));
    await tester.pump();
    expect(find.text('Score: 80'), findsOneWidget);
  });
```

```
}
```

TERMINAL — run checks

```
dart format lib test
flutter analyze
flutter test
flutter run
```

How the solution works. A pure Dart function checks bounds and returns a deterministic result, making it easy to test independently from rendering. The widget test pumps ScorePage, taps the button and asserts visible output. flutter test reports failures before the app is deployed.

Expected output. Two unit tests and one widget test pass; the phone app updates the visible score from 0 to 80.

Check your work. Ensure your flutter create project name really is mad_lab13, because the package imports in test/ match the generated pubspec.yaml name.

Dependency inversion and design reasoning

High-level business rules should depend on contracts, not low-level HTTP/database details. The ViewModel in Task 1 knows only CourseRepository; it does not know whether entries came from an API, Drift or a fake repository. For larger apps, consider a dependency container, typed error objects and immutable view state, but avoid adding complexity without a concrete need.

Scenario-based practice — unsolved

Scenario 1. Hospital queue architecture. Separate queue screen, queue ViewModel, server repository and cache repository. Explain which parts can be unit-tested without an Android phone.

Scenario 2. Fault injection. Your network repository intermittently fails and returns duplicate records. Create fake responses for latency, HTTP failure, empty data and duplicates; define UI states and acceptance tests.

Technical viva — justify your reasoning

1. How does constructor injection reduce coupling?
2. What is the difference between a repository and a ViewModel?
3. Why is a pure function easier to unit-test?
4. What does pumpWidget create, and what does pump rebuild?
5. How would you test a ViewModel that relies on an asynchronous repository?
6. What behavior justifies a mock/fake data source rather than a live backend test?

Submission and self-check

Submit architecture folder tree, tests showing PASS, screenshot of the app and the output of flutter analyze. Assignment checkpoint 5 can use these artifacts.

CSC303 / NOVEMBER 2026 REVISION

LAB 13 EXTENSION | Unit-test the repository/ViewModel boundary

Run this extension in a copy of **Task 1** that still contains `lib/data/course_repository.dart` and `lib/viewmodel/course_viewmodel.dart`. Do not add it to the unrelated score-screen Task 2 replacement.

New file: test/course_viewmodel_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:mad_lab13/data/course_repository.dart';
import 'package:mad_lab13/viewmodel/course_viewmodel.dart';

class FakeRepo implements CourseRepository {
  final bool shouldFail;
  FakeRepo({this.shouldFail = false});
  @override
  Future<List<String>> loadCourses() async {
    if (shouldFail) throw StateError('test outage');
    return ['Mobile App Development', 'Operating Systems'];
  }
}

void main() {
  test('loads and reports courses', () async {
    final vm = CourseViewModel(FakeRepo());
    final transitions = <bool>[];
    vm.addListener(() => transitions.add(vm.loading));
    await vm.load();
    expect(transitions, [true, false]);
    expect(vm.courses, hasLength(2));
    expect(vm.error, isNull);
    vm.dispose();
  });
  test('reports repository error without staying busy', () async {
    final vm = CourseViewModel(FakeRepo(shouldFail: true));
    await vm.load();
    expect(vm.loading, isFalse);
    expect(vm.error, contains('test outage'));
    vm.dispose();
  });
}
```

If you created a different package name, change both import prefixes to match `pubspec.yaml`. These tests demonstrate dependency inversion directly: no HTTP request, database, Firebase project or phone is required.

Run and explain

```
dart format lib test
flutter analyze
flutter test test/course_viewmodel_test.dart
```

- Explain the fake repository and why it is deterministic.
- Show error and successful-load state transitions and how the UI listens for notifications.
- For a more advanced submission, add a pending Future to prove the loading indicator remains active while the repository has not completed.

LAB 14 Rendering performance, animations and Android native channels

Unit 5 • 3-hour guided laboratory • CLO-5

Learning goal. Use a controlled animation without resource leaks, construct performant scrollable widgets and obtain real Android hardware information through a MethodChannel.

Core concepts. AnimatedContainer, AnimationController, SingleTickerProviderStateMixin, ListView.builder, debug profiling, Kotlin/Flutter platform channels.

Before you start. Keep a physical Android device attached. Task 2 requires a specifically named Kotlin-package Android project; see its command.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab14
cd mad_lab14
```

Solved task 1 | AnimationController with animated card and efficient list

Problem and objective. Make a button transition the size and color of a banner while an explicit AnimationController animates a progress line. Render a lazy 1000-row list.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: AnimatedPage()));

class AnimatedPage extends StatefulWidget {
  const AnimatedPage({super.key});
  @override
  State<AnimatedPage> createState() => _AnimatedPageState();
}

class _AnimatedPageState extends State<AnimatedPage>
    with SingleTickerProviderStateMixin {
  late final AnimationController controller;
  bool expanded = false;

  @override
  void initState() {
    super.initState();
    controller = AnimationController(
      vsync: this, duration: const Duration(milliseconds: 500));
  }

  @override
  void dispose() { controller.dispose(); super.dispose(); }

  void toggle() {
    setState(() => expanded = !expanded);
    if (expanded) { controller.forward(); }
    else { controller.reverse(); }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Flutter animation lab')),
    body: Column(children: [
      AnimatedContainer(
        duration: const Duration(milliseconds: 500),
        margin: const EdgeInsets.all(12),
        height: expanded ? 140 : 75,
        width: double.infinity,
        decoration: BoxDecoration(
          color: expanded ? Colors.teal : Colors.indigo.shade200,
          borderRadius: BorderRadius.circular(14)),
        child: const Center(child: Text('Tap below to animate')),
      ),
      AnimatedBuilder(animation: controller,
        builder: (_, __) => LinearProgressIndicator(
          value: controller.value)),
      FilledButton(onPressed: toggle,
        child: Text(expanded ? 'Collapse' : 'Expand')),
      Expanded(child: ListView.builder(
```

```
        itemCount: 1000,
        itemBuilder: (_, i) => ListTile(
          leading: const Icon(Icons.bolt),
          title: Text('Efficient row ${i + 1}')),
      )),
    ]),
  );
}
```

How the solution works. AnimatedContainer interpolates its properties implicitly when the state changes. AnimationController advances an explicit numeric progress value, and AnimatedBuilder rebuilds only its progress subtree. The ticker and controller are disposed to prevent leaked animation resources. ListView.builder lazily builds visible rows, making 1000 records feasible.

Expected output. The card expands and changes color; a progress bar animates in sync and the large list scrolls.

Check your work. Use Flutter DevTools Performance and test in profile mode on a real phone if you investigate frame timing; debug-mode timing is not representative.

Solved task 2 | Kotlin MethodChannel to read battery percentage

Problem and objective. Create a Flutter project with an exact Android package name so the native file path matches. A Dart MethodChannel requests the battery percentage from Kotlin's BatteryManager.

TERMINAL — NEW PROJECT (run outside mad_lab14)

```
flutter create --platforms=android --org edu.cui.sahiwal mad_native_demo
cd mad_native_demo
# Open this folder in Android Studio before editing MainActivity.kt
```

android/app/src/main/kotlin/edu/cui/sahiwal/mad_native_demo/MainActivity.kt

```
package edu.cui.sahiwal.mad_native_demo

import android.content.Intent
import android.content.IntentFilter
import android.os.BatteryManager
import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.plugin.common.MethodChannel

class MainActivity : FlutterActivity() {
    private val channel = "edu.cui.sahiwal/device"

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger, channel)
            .setMethodCallHandler { call, result ->
                if (call.method == "batteryLevel") {
                    val battery = registerReceiver(
                        null, IntentFilter(Intent.ACTION_BATTERY_CHANGED))
                    val level = battery?.getIntExtra(
                        BatteryManager.EXTRA_LEVEL, -1) ?: -1
                    val scale = battery?.getIntExtra(
                        BatteryManager.EXTRA_SCALE, -1) ?: -1
                    if (level >= 0 && scale > 0) {
                        result.success(level * 100 / scale)
                    } else {
                        result.error("UNAVAILABLE", "Battery not available", null)
                    }
                } else {
                    result.notImplemented()
                }
            }
    }
}
```

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

void main() => runApp(const MaterialApp(home: BatteryPage()));

class BatteryPage extends StatefulWidget {
  const BatteryPage({super.key});
  @override
  State<BatteryPage> createState() => _BatteryPageState();
}
```

```
class _BatteryPageState extends State<BatteryPage> {
  static const channel = MethodChannel('edu.cui.sahiwal/device');
  String battery = 'Not read yet';

  Future<void> readBattery() async {
    try {
      final value = await channel.invokeMethod<int>('batteryLevel');
      if (mounted) setState(() => battery =
          value == null ? 'Unknown' : '$value%');
    } on PlatformException catch (e) {
      if (mounted) setState(() => battery = 'Native error: ${e.code}');
    } on MissingPluginException {
      if (mounted) setState(() => battery = 'Native channel missing');
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Android battery channel')),
    body: Center(child: Column(mainAxisSize: MainAxisSize.min,
      children: [
        Text('Battery: $battery',
          style: Theme.of(context).textTheme.headlineSmall),
        FilledButton(onPressed: readBattery,
          child: const Text('Read Android battery')),
      ],
    )),
  );
}
```

TERMINAL — native code requires full rebuild

```
flutter clean
flutter pub get
flutter run
```

How the solution works. MethodChannel communicates between the Dart UI and the Android embedding through a string channel name and a method name. Kotlin reads the sticky ACTION_BATTERY_CHANGED broadcast and translates the level/scale pair into a percentage. A native source edit requires a complete rebuild; Dart hot reload does not replace compiled Kotlin.

Expected output. The phone reports a battery percentage close to Android settings; no dangerous runtime permission is necessary for this broadcast.

Check your work. Check that the package statement matches the directory and Gradle application namespace, and that both sides spell channel/method identically.

Scenario-based practice — unsolved

Scenario 1. Animated attendance tile. Design a reduced-motion-safe attention animation for late arrivals. Explain why repeating animations may be distracting and how to honor accessibility settings.

Scenario 2. Hardware feature request. A campus app must read accelerometer values and upload aggregated readings. Identify permission, privacy, platform-interface, battery and sampling concerns.

Technical viva — justify your reasoning

1. Why does AnimationController need a vsync provider?
2. How is a repaint different from a build and layout pass?
3. Why is profile mode better than debug mode for benchmarking?
4. What causes MissingPluginException?
5. Why can Dart hot reload not apply modified Kotlin bytecode?
6. What is the impact of a channel name mismatch?

Submission and self-check

Submit a smooth animation clip, a battery reading on a real Android device, Kotlin and Dart source, and a note on profiling versus debugging.

CSC303 / NOVEMBER 2026 REVISION

LAB 14 EXTENSION | Evidence-based performance and native bridge

Students should distinguish a visual animation demonstration from a performance measurement. Running at 60 Hz is not proof that a screen maintains its frame budget under stress.

Performance experiment

• Run the animation demo on a real phone in profile mode. Record device model, screen refresh rate, Flutter version, and which performance tool was used.

```
flutter devices
flutter run --profile -d ANDROID_DEVICE_ID
# Inspect frame timings in Flutter DevTools Performance.
```

• Capture a trace with the animation running, then another after reducing unnecessary rebuilds. Compare representative frame timings; do not invent FPS values.

• Turn on device reduced-motion preference and verify that distracting repeated motion is disabled or replaced by an accessible representation.

MethodChannel correctness checks

• Test the exact channel identifier and method identifier on both the Kotlin and Dart sides.

• Verify `level` and `scale` are present and a positive scale is used before computing a battery percentage.

• Change native Kotlin code; perform a full rebuild rather than relying on Dart hot reload.

• Demonstrate a graceful failure if a native plugin/channel is missing, without an uncaught crash.

Optional extension

Wrap the platform-channel call behind a small interface. Create a fake in a unit test that returns 42 and one that throws an error. This illustrates how architecture and tests from Lab 13 transfer to Android integration.

Submission

Provide one annotated profiling screenshot, one accessibility demonstration, and the Dart/Kotlin source with reproducible device and SDK details.

LAB 15 Integrated capstone: API, offline persistence and multiple screens

Unit 2–5 • 3-hour guided laboratory • CLO-5

Learning goal. Combine remote data, persistent state, Android navigation and error handling into a small deliverable, then plan a larger team semester project against CLO-5.

Core concepts. Integration testing; JSON API + cache + favorites; offline tasks; cross-screen data flow; reproducible Android APK; issue-driven project delivery.

Before you start. Successful completion of Labs 01–14. For Task 1 ensure Android INTERNET permission from Lab 10 is inserted and install packages.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab15
cd mad_lab15
flutter pub add http shared_preferences
```

Solved task 1 | Offline article feed, bookmarks and details

Problem and objective. Build a functioning feed that fetches titles and bodies from JSONPlaceholder, caches the last successful response, retains favorite article IDs and navigates to a detail page. Add Android INTERNET permission as shown in Lab 10.

lib/main.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';

class Article {
  final int id;
  final String title;
  final String body;
  Article(this.id, this.title, this.body);
  factory Article.fromJson(Map<String, dynamic> x) => Article(
    x['id'] as int, x['title'] as String, x['body'] as String);
}

void main() => runApp(const MaterialApp(home: ArticleFeed()));

class ArticleFeed extends StatefulWidget {
  const ArticleFeed({super.key});
  @override
  State<ArticleFeed> createState() => _ArticleFeedState();
}

class _ArticleFeedState extends State<ArticleFeed> {
  final prefs = SharedPreferencesAsync();
  List<Article> articles = [];
  Set<String> favorites = {};
  String status = 'Loading...';
  bool busy = false;

  @override
  void initState() { super.initState(); load(); }

  Future<void> load() async {
    if (busy) return;
    setState(() => busy = true);
    var source = 'Live API';
    String raw;
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts'))
          .timeout(const Duration(seconds: 10));
      if (response.statusCode != 200) throw Exception('HTTP error');
```

```
      raw = response.body;
      final parsed = jsonDecode(raw) as List<dynamic>;
      parsed.take(25).map((e) => Article.fromJson(
        e as Map<String, dynamic>)).toList();
      await prefs.setString('feed_cache', raw);
    } catch (_) {
      raw = await prefs.getString('feed_cache') ?? '[]';
      source = raw == '[]' ? 'Offline, no data' : 'Offline cache';
    }
    try {
      final decoded = jsonDecode(raw) as List<dynamic>;
      final values = decoded.take(25).map((e) =>
        Article.fromJson(e as Map<String, dynamic>)).toList();
      final saved = (await prefs.getStringList('favs')) ?? [];
      if (mounted) setState(() {
        articles = values; favorites = saved.toSet(); status = source;
      });
    } catch (_) {
      if (mounted) setState(() => status = 'Invalid response / cache');
    } finally {
      if (mounted) setState(() => busy = false);
    }
  }
```

```
  Future<void> toggle(int id) async {
    final next = {...favorites};
    if (!next.add('$id')) next.remove('$id');
    await prefs.setStringList('favs', next.toList());
    if (mounted) setState(() => favorites = next);
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('CSC303 article reader')),
    body: Column(children: [
      ListTile(title: Text(status),
        subtitle: Text('${favorites.length} favorites'),
        trailing: IconButton(onPressed: busy ? null : load,
          icon: const Icon(Icons.refresh))),
      if (busy) const LinearProgressIndicator(),
      Expanded(child: ListView.builder(
        itemCount: articles.length,
        itemBuilder: (context, i) {
          final article = articles[i];
          final saved = favorites.contains('${article.id}');
          return ListTile(
            title: Text(article.title),
            leading: CircleAvatar(child: Text('${article.id}')),
            trailing: IconButton(
              icon: Icon(saved ? Icons.bookmark : Icons.bookmark_border),
              onPressed: () => toggle(article.id)),
            onTap: () => Navigator.push(context, MaterialPageRoute(
              builder: (_) => ArticleDetail(article: article))),
          );
        },
      )),
    ]),
  );
}

class ArticleDetail extends StatelessWidget {
  final Article article;
  const ArticleDetail({super.key, required this.article});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: Text('Article ${article.id}')),
    body: ListView(padding: const EdgeInsets.all(20), children: [
      Text(article.title,
        style: Theme.of(context).textTheme.headlineSmall),
      const SizedBox(height: 12),
      Text(article.body),
    ]),
  );
}
```

How the solution works. The feed retrieves typed data from a public API, writes raw successful responses to cache, then parses and displays them. A set of bookmark IDs is serialized through SharedPreferencesAsync. The details page takes an immutable Article by constructor. Error state and offline/source labels expose whether the content is fresh.

Expected output. Online: 25 articles, bookmarking icons and detail navigation. Offline after a successful refresh: cached articles and existing bookmarks remain.

Check your work. Start online, bookmark two IDs, close app, enable airplane mode, reopen, navigate to a detail and verify two favorites persist.

Solved task 2 | Offline task board with a separate details page

Problem and objective. Create a personal sprint board where tasks can be added and checked off, stored as JSON locally, and opened on a detail route. This has no online dependency and works on a phone immediately after shared_preferences is installed.

lib/main.dart — replace all of Task 1

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

class WorkItem {
  String title;
  bool completed;
  WorkItem(this.title, this.completed);
  Map<String, dynamic> toJson() =>
    {'title': title, 'completed': completed};
  factory WorkItem.fromJson(Map<String, dynamic> value) => WorkItem(
    value['title'] as String, value['completed'] as bool);
}

void main() => runApp(const MaterialApp(home: TaskBoard()));

class TaskBoard extends StatefulWidget {
  const TaskBoard({super.key});
  @override
  State<TaskBoard> createState() => _TaskBoardState();
}

class _TaskBoardState extends State<TaskBoard> {
  final prefs = SharedPreferencesAsync();
  final tasks = <WorkItem>[];

  @override
  void initState() { super.initState(); restore(); }

  Future<void> restore() async {
    final text = await prefs.getString('sprint_tasks');
    if (text == null) return;
    try {
      final rows = jsonDecode(text) as List<dynamic>;
      if (mounted) setState(() {
        tasks..clear()..addAll(rows.map((x) =>
          WorkItem.fromJson(x as Map<String, dynamic>)));
      });
    } catch (_) { /* Ignore corrupted demo data. */ }
  }

  Future<void> save() async {
    await prefs.setString('sprint_tasks',
      jsonEncode(tasks.map((x) => x.toJson()).toList()));
  }

  Future<void> addTask() async {
    String draft = '';
    final value = await showDialog<String>(context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('New sprint task'),
        content: TextField(autofocus: true,
```

```
          onChanged: (value) => draft = value,
          decoration: const InputDecoration(hintText: 'Task title')),
        actions: [
          TextButton(onPressed: () => Navigator.pop(dialogContext),
            child: const Text('Cancel')),
          FilledButton(onPressed: () =>
            Navigator.pop(dialogContext, draft.trim()),
            child: const Text('Add')),
        ],
      ));
    if (value == null || value.isEmpty || !mounted) return;
    setState(() => tasks.add(WorkItem(value, false)));
    await save();
  }

  @override
  Widget build(BuildContext context) {
    final finished = tasks.where((x) => x.completed).length;
    return Scaffold(
      appBar: AppBar(title: Text('Sprint board • $finished/${tasks.length}')),
      floatingActionButton: FloatingActionButton(
        onPressed: addTask, child: const Icon(Icons.add)),
      body: ListView.builder(
        itemCount: tasks.length,
        itemBuilder: (context, i) {
          final item = tasks[i];
          return CheckboxListTile(
            value: item.completed,
            title: Text(item.title),
            subtitle: Text(item.completed ? 'Done' : 'Open'),
            secondary: IconButton(
              icon: const Icon(Icons.open_in_new),
              onPressed: () => Navigator.push(context, MaterialPageRoute(
                builder: (_) => TaskDetail(item: item)))),
            onChanged: (v) {
              setState(() => item.completed = v ?? false); save();
            },
          );
        },
      ),
    );
  }
}

class TaskDetail extends StatelessWidget {
  final WorkItem item;
  const TaskDetail({super.key, required this.item});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Task detail')),
    body: Padding(padding: const EdgeInsets.all(20),
      child: Column(crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(item.title,
            style: Theme.of(context).textTheme.headlineSmall),
          Text('Current status: ${item.completed ? 'Done' : 'Open'}'),
        ],
      )),
  );
}
```

How the solution works. Each task is a model object with JSON serialization. The board stores a complete JSON list and restores it when initialized. Checkbox state is changed in memory before persistence. The detail route receives the object and shows current status; this is a small integrated example, not a multi-user synchronized task system.

Expected output. Students can add tasks, complete them, open details and reopen the app to see the saved sprint state.

Check your work. Test a brand-new install, three additions, two completions, force-close, restore and one canceled add dialog.

Final semester project — student brief (50 marks)

Build a medium-sized Android Flutter application for a genuine campus or community need (e.g., timetable and announcements, library reservations, hostel complaint tracking, semester planner, food ordering prototype, or campus shuttle monitoring). The app must be distinguishable from these worked examples and should demonstrate the CDF's CLO-5.

Mandatory requirement	Acceptance evidence
Multiple screens	At least three meaningful screens with coherent navigation and state transfer.
User interface	Responsive and accessible Material UI, form validation, loading/empty/error states.
Online API	At least one meaningful HTTPS endpoint with parsed typed data and error handling.
Persistent storage	SQLite/Drift or Firestore plus an explicit offline or persisted-use case.
Architecture	Recognizable presentation, business logic and repository separation.
Version control	Issue board, feature branches, minimum two reviewed PRs for a team, useful commits.
Verification	Automated tests and documented physical Android testing, flutter analyze passing.
Delivery	GitHub repository, README, report, screens, demo and installable debug/release APK.

Project milestones and demonstration protocol

Proposal: one page containing problem, target users, data model, screen flow, API selection, threat/privacy considerations and issue plan. Sprint 1: navigation and functional UI. Sprint 2: persistence/API and failure handling. Sprint 3: architecture refactoring and automated tests. Final demo: install on a phone from a clean checkout, run the critical journey, deliberately disable networking, show locally retained state and explain the Git contributions of every group member.

Scenario-based practice — unsolved

Scenario 1. Capstone: shuttle tracker. Design a campus shuttle app with live arrivals, saved routes and a useful offline fallback. Specify screen map, source of truth, stale data indicator and fair team ownership.

Scenario 2. Capstone: complaint system. Students submit complaints with optional images and staff update status. Explain access control, upload safety, offline edits, conflict resolution and audit logs.

Technical viva — justify your reasoning

1. What evidence proves an app is offline-capable instead of just remembering UI state in RAM?
2. What data should be the source of truth after reconnecting?
3. How would you prevent duplicate writes when a retry follows a timeout?
4. How do you show ownership and code-review evidence in a three-person project?
5. Which parts of a Flutter app should have unit tests vs integration tests?
6. How should a developer handle migrations after releasing version 1 of an app?

Submission and self-check

Submit a stable final demo, documented Android APK, repo/PR links, tests, API/offline screenshots and the 50-mark project rubric evidence.

CSC303 / NOVEMBER 2026 REVISION

LAB 15 EXTENSION | From worked examples to an assessable app

Scope clarification: Original Task 1 is a network-first article reader with a SharedPreferences cache; Task 2 is a local task board. They are valid practice components, but neither alone fulfills the complete 50-mark final project specification later in Lab 15.

Reference architecture to build for the final project

```
semester_project/
  lib/
    main.dart                    # app, theme, routes
    models/article.dart          # typed data + validation
    data/local/app_database.dart # Drift + schema/migrations
    data/remote/article_api.dart # injectable HTTP client
    data/article_repository.dart # coordinates local/remote
    viewmodels/feed_viewmodel.dart
    screens/feed_screen.dart
    screens/detail_screen.dart
    screens/saved_screen.dart
  test/
    article_repository_test.dart
    feed_viewmodel_test.dart
    widget_test.dart
  README.md
```

Start by integrating the Lab 15 article model and network code with the Lab 09 Drift table. Use a repository to expose a local stream as the screen's data source; keep remote synchronization separate. This is an architectural exercise, not a drop-in replacement for Lab 15's complete `main.dart`.

Data flow and failure handling

```
Android screens -> ViewModel -> Repository
                            |            |
                            v            v
                      Drift/SQLite    HTTPS remote
                            ^            |
                            +-- validated, mapped records

On app start: read local rows immediately.
On refresh: fetch/parse/validate -> database transaction.
On failure: keep local rows; show stale + retry status.
On favorites: save change locally; verify after restart.
```

Integration checkpoints

Stage	Observable outcome
A. UI/navigation	Feed, detail and saved screens; errors/empty/loading states
B. Local persistence	Drift table, insert/update/delete, restart proof
C. Remote import	Typed HTTPS result, timeout/error behavior, safe local update
D. Offline flow	Airplane mode launch shows prior rows; no in-memory illusion
E. Testability	Fake remote/local data tests, ViewModel test and widget test
F. Delivery	Clean checkout, regenerated code, report, two PRs if team

CSC303 / NOVEMBER 2026 REVISION

FINAL PROJECT EXTENSION | Detailed 50-mark rubric

This detailed matrix elaborates the existing page-4 categories while preserving every published category weight. Assessors should score observable evidence rather than screenshots alone.

Category	Marks	Specific evidence
Functionality	15	Core end-to-end user journey (7); three meaningful screens (4); validation, navigation/back actions (4)
API and persistence	10	Meaningful HTTPS parsing and error paths (4); Drift/Firestore persistence (4); offline restart proof (2)
UI and accessibility	7	Responsive layouts (2); large-text/screen-reader checks (2); clear loading/empty/error states (3)
Architecture and tests	6	ViewModel/repository separation (2); meaningful automated tests (3); reproducible clean build (1)
Git process	5	Issue/branch traceability (2); at least two reviewed team PRs (2); contribution explanation (1)
Demo and viva	7	Clean-device critical journey (3); forced-offline demo (2); technical reasoning (2)

Demonstration script and acceptance tests

• From a clean checkout run `flutter pub get`, any required code generation, `flutter analyze`, and project tests. Document generated configuration steps; do not publish secrets.

• On a real Android phone, navigate through three meaningful screens and exercise a primary user action.

• Acquire and display typed data from a real HTTPS service. Show one failed request and an understandable UI message.

• Restart the app with network disabled; show persistent data retrieved from a local database or supported Firestore offline mode, not merely retained in a live widget.

• Open issue/PR history and identify what each teammate contributed and reviewed.

Independent work within a team

Team submissions must include a responsibility matrix and an individual viva. A team's shared functionality does not automatically prove equal participation. Where collaboration is unavailable, the instructor should publish an individual equivalent for the Git-process marks.

Explicit exclusions

The Firebase Storage/FCM demo from Lab 12 is optional unless the chosen project requires it and the instructor has approved service access. Students are not required to incur paid cloud charges to demonstrate CLO-5.

Appendix A | Reproducible execution checklist

Perform this checklist before requesting a lab evaluation. A screenshot is not a substitute for a working build.

- Use flutter --version and flutter doctor -v to record actual Flutter/Android tooling versions.
- Use adb devices and flutter devices; verify a physical Android phone reports status device and appears as an Android target.
- Execute flutter pub get after dependency edits; run code generation before running Drift examples.
- Run dart format . and flutter analyze. Resolve all compile errors; review lint warnings.
- Run flutter test only with tests written for the CURRENT app. Flutter create generates test/widget_test.dart for its starter widget (often MyApp); after you replace lib/main.dart the old starter test will not compile. Replace it with a relevant test as shown in Lab 01 or Lab 13, or remove the obsolete file before invoking flutter test. Never treat a failing starter test as an app failure.
- Run flutter run on Android; exercise normal path, empty state, error state, navigation back and restart persistence.
- Inspect git status; commit source and tests but exclude .env, private keys, build artifacts and local Android signing keystores.
- Confirm README instructions work in a fresh cloned checkout and attach a link to the latest reviewed PR.

Appendix B | Troubleshooting reference

Symptom	Likely diagnosis / corrective action
flutter: command not found	Add real Flutter SDK bin directory to PATH; restart terminal; IDE plugin alone is insufficient.
Android toolchain not found	Install an Android SDK Platform, Command-line Tools, Platform-Tools and Build-Tools via SDK Manager; flutter doctor -v.
Android licenses missing	Run flutter doctor --android-licenses, accept licenses, rerun doctor.
adb devices: unauthorized	Unlock phone; approve USB debugging fingerprint; revoke/re-approve permissions if necessary.
adb devices: no device	Check data cable/MTP, OEM USB driver (Windows), developer mode, SDK platform tools and phone settings.
Device shown but build fails	Read the first Gradle/Kotlin error, check SDK level and internet/space; use flutter clean only if cache is suspected.
NoSuchMethod / JSON cast error	Check actual HTTP response shape and null fields; ensure status == 200 and decode typed payload safely.
database.g.dart missing	Run dart run build_runner build --delete-conflicting-outputs with drift_dev/build_runner installed.
Firebase app not configured	Run flutterfire configure for this exact Android project, verify generated firebase_options.dart and service activation.
permission-denied Firestore	Check current Firebase UID, rules and document path; do not disable rules to mask the problem.
Cloud Storage billing / bucket failure	Confirm institutional Blaze/bucket approval; otherwise do not run the paid Storage operation.
MissingPluginException on Kotlin	Check channel name, file package path and full rebuild; hot reload is insufficient.
GitHub Permission denied (publickey)	Check public key registered on GitHub, ssh-agent loaded private key, remote SSH URL and ssh -T output.
flutter test refers to MyApp	Replace/remove template widget_test.dart or update the test to use the classes in the new example.

Appendix C | Lab report template

COPY-PASTE SUBMISSION TEMPLATE

```
CSC303 – Lab report
Student name / registration number:
Lab number and title:
GitHub repository / exact commit SHA:
Physical Android device (OS and model):
Flutter version / Android SDK info:

1. Learning objective (2–3 sentences):
2. Implementation files and packages:
3. Key algorithm or architecture (100–150 words):
4. Reproducible setup commands:
5. Test cases with expected and actual results:
6. Screenshots (normal, empty/error, Android phone):
7. Viva preparation – answer at least two questions:
8. Observed problems and resolution:
9. Attribution / collaborators / reviewed PR links:
10. Academic integrity declaration:
```

Appendix D | Git & Android command reference

REFERENCE COMMANDS (some require being inside a Git / Flutter project)

```
# Git history and collaboration
git status
git diff
git diff --cached
git log --oneline --graph --decorate -8
git branch -a
git remote -v
git fetch origin
git switch -c feat/new-feature
git add lib test
git commit -m "feat: implement new feature"
git push -u origin feat/new-feature

# Flutter validation and build
flutter doctor -v
flutter devices
flutter pub get
dart format .
flutter analyze
flutter test
flutter run
flutter build apk --release

# Android device tools
adb devices
adb shell getprop ro.build.version.release
adb logcat -d -t 200
```

Appendix E | CLO and coverage map

CDF outcome / unit	Where demonstrated
CLO-1 / CDF Unit 1	Labs 01–02: platform/tooling, IDE, project structure and configuration.
CLO-2 / CDF Units 2–3	Labs 03–07 and 14: responsive widgets, UI events, navigation, theme, animations.
CLO-3 / CDF Unit 4	Labs 09–12 and 15: relational/local data, JSON APIs, caching and cloud backend.
CLO-4 / CDF Unit 5	Labs 08, 13 and 15: state ownership, repositories, MVVM and testing.
CLO-5 / CDF lab units 2–5	Assessed through integrated Labs 10–15 and the semester

CDF outcome / unit	Where demonstrated
	project with API + persistence.

Appendix F | Technical glossary

- Widget tree — nested declarative UI description used by Flutter to update the rendered scene.
- Build / layout / paint — successive phases that describe widgets, compute geometry and draw pixels.
- State — information that may change while the app runs; local ephemeral state differs from app-wide or persisted state.
- Repository — data access abstraction through which an app reads and writes local/remote data.
- Asynchronous operation — work completing in the future via Future or repeatedly emitting values through Stream.
- Offline-first — design that remains useful without network connectivity and explicitly reconciles/stales remote data.
- SSH key pair — private/public cryptographic credentials used for secure Git remote authentication.
- Pull request — proposed change to a repository branch that can be reviewed, tested and merged.
- Drift code generation — source transformation that generates typed classes and database access APIs from schema declarations.
- FCM — Firebase Cloud Messaging; a transport for notifications/data payloads, distinct from Cloud Firestore data storage.
- MethodChannel — Flutter interface to native platform code (such as Android Kotlin) through named method calls.

Appendix G | Official documentation and verification sources

The technology guidance in this edition was checked against the official pages below in October 2026. Flutter/Android/Firebase versions, pricing, API requirements and UI labels can change; inspect current vendor documentation whenever lab environment behavior differs.

Flutter — install SDK manually — https://docs.flutter.dev/install/manual

Flutter — setup Android development and physical devices — https://docs.flutter.dev/platform-integration/android/setup

Flutter — installation troubleshooting — https://docs.flutter.dev/install/troubleshoot

Flutter — networking recipes — https://docs.flutter.dev/cookbook/networking/fetch-data

Flutter — state management with Provider — https://docs.flutter.dev/data-and-backend/state-mgmt/simple

Dart language documentation — https://dart.dev/language

Android Studio — SDK Manager and updates — https://developer.android.com/studio/intro/update

Android Studio official download — https://developer.android.com/studio

GitHub — new SSH key and ssh-agent — https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent

GitHub — register SSH public key — https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account

GitHub — testing SSH connection — https://docs.github.com/en/authentication/connecting-to-github-with-ssh/testing-your-ssh-connection

FlutterFire — official project configuration — https://firebase.google.com/docs/flutter/setup

Firebase — email/password authentication — https://firebase.google.com/docs/auth/flutter/password-auth

Firebase — Cloud Storage requirements — https://firebase.google.com/docs/storage/flutter/start

Firebase — FCM Flutter codelab — https://firebase.google.com/codelabs/firebase-fcm-flutter

Drift — current setup and code generation — https://drift.simonbinder.eu/setup/

shared_preferences — current API — https://pub.dev/packages/shared_preferences

image_picker — current API — https://pub.dev/packages/image_picker

flutter_bloc — package documentation — https://pub.dev/packages/flutter_bloc

Final student declaration

I confirm that the submitted work is mine or clearly identified team work, that reused code is appropriately acknowledged, that I have not submitted any other student's work as my own, and that the presented screenshots and test evidence reflect actual program execution. I understand that secret credentials must never be committed to GitHub or submitted with the report.

Student signature: _____ Date: _____

CSC303 / NOVEMBER 2026 REVISION

APPENDIX | Release verification and change record

This page distinguishes the revision's editorial validation from a full clean-room Android build. The manual should be treated as reviewed teaching material until all rows below are completed in an instructor-controlled environment.

Verification	Required evidence	Status at PDF publication
PDF structure and page layout	Pages render and contain selectable text	CHECKED
Lab 12 dependency command	Correct shell invocation and package resolution	INSTRUCTION CORRECTED; SDK RUN PENDING
Lab 09 update extension	Drift generated code, analyze and phone run	STATIC; SDK RUN PENDING
Lab 10/15 cache test	Malformed HTTP 200 preserves good cache	STATIC; TEST RUN PENDING
Lab 11/12 Firebase	Auth, rules, messaging and permitted billing	EXTERNAL CONFIGURATION REQUIRED
Lab 13 fake-repository test	flutter test passes in Task 1 project	STATIC; SDK RUN PENDING
All 15 worked lab samples	Analyze/test/run on approved Android baseline	NOT YET CERTIFIED

Nov 2026 revision change record

- Fixed the Lab 12 multi-line dependency installation defect and clarified cloud service cost/security prerequisites.
- Corrected Lab 09 references that implied full CRUD without an update implementation.
- Corrected Lab 15 cache-overwrite ordering so response data is validated before storage.
- Clarified the Lab 01 training repository versus the long-term portfolio repository structure.
- Added API failure-test matrix, Firebase auth refresh, ViewModel fake tests, accessibility protocol, timed assessment evidence and an integrated final-project plan.
- Preserved original assessment weighting and 15-lab sequence; added detailed criteria only as an assessment aid.

References for maintenance

Flutter Android setup (official)
Flutter testing (official)
Drift setup (official)
Shared preferences package
Firebase Storage / billing (official)
Flutter FCM delivery (official)

For the next edition, record the exact Flutter version, pubspec.lock state and last successful clean build commit alongside this change record.